Supplementary material

VCF-RDFizer: From a VCF File to Explicit, Verifiable, and Policy-aware Semantic Genomic Data

Elias Crum, Bart Buelens, Gökhan Ertaylan†, Ruben Taelman†

†These authors share final authorship and jointly supervised this work.

The supplement provides the implementation, inputs, protocols, and complete measurements supporting the main text. An interactive results browser, built from the same run records, presents these measurements by research question at `https://ecrum19.github.io/vcf-rdfizer-testing/`. Completed comparisons, expected refusals (option combinations the tool is designed to reject, such as the single HDT strategy with COTTAS), incomplete checks, and unresolved limits are reported as separate outcomes, and Section S8 collects the defects and remaining gaps. The linked workflow evaluates prerequisites for the motivating applications rather than complete clinical or governance systems.

Contents

S1 Documented capabilities of related tools **3**

S2 Shared-input converter comparison **5**

S2.1 Converters, builds, and options . . . 5

S2.2 Shared validation protocol and query adaptation . . . 6

S2.3 Results . . . 6

S3 Representation and implementation details **7**

S3.1 Model and sample profiles . . . 7

S3.2 Conversion and incremental representation construction . . . 10

S4 Validation specification, scope, and sensitivity **10**

S4.1 Queries, normalization, and engines . . . 10

S4.2 Completed comparisons and the consumer WGS validation run . . . 11

S4.3 Fault sensitivity and oracle limits . . . 12

S5 Linked genomic workflow: protocol, policies, and detailed results **13**

S5.1 Scope, linking, and policy evaluation . . . 13

S5.2 Application question, comparison workflows, and evaluation scenarios . . . 14

S5.3 Linking results, matches, and rule changes . . . 15

S5.4 Stage costs . . . 18

S6 Reproducibility and execution **20**

S6.1 The core benchmark . . . 20

S6.2 Builds and dependencies . . . 21

S6.3 Hosts, measurements, and execution status . . . 24

S7 Input and version coverage **24**

S8 Defects, corrections, and remaining gaps **25**

S9 Conversion, storage, and retrieval measurements **27**

S9.1 Record and sample scaling . . . 27

S9.2 Storage mode and optional artifacts 28
S9.3 Query workloads and physical access paths 29
S9.4 Setup amortization 31
S10 Configuration coverage, operating modes, and memory feasibility 33

S1 Documented capabilities of related tools

Table S1 summarizes the documented capabilities discussed in the main text and gives the evidence behind each mark. The assessment used repositories, manuals, cited papers, and package registries inspected in September 2026 and rechecked in October 2026. The marks describe documented capabilities, not measurements on shared inputs. An open mark indicates that a capability was not documented, not that the capability is impossible. For superseded tools, the assessment used the version that reads VCF. The following definitions explain each column and the criteria for a partial mark.

RDF model The output uses a documented vocabulary. A partial mark indicates terms shown only by example or a vocabulary that is no longer published.

Header + genotype The RDF retains header metadata together with per-sample genotype and FORMAT data. A partial mark indicates only header or genotype coverage, or combined coverage available only through user-written templates.

Editable mappings Users can change the VCF-to-RDF mapping without modifying the converter. A partial mark indicates configuration or plug-in hooks over a fixed mapping, or mappings that generate only part of the output.

CLI The tool documents command-line use suitable for pipelines. A partial mark indicates only library, browser, or user-interface access.

Reproducible workflow The tool documents a VCF-to-RDF workflow that a third party can rerun. A partial mark indicates manual installation of several components, an unpublished package, an offline service, or an unmaintained runtime.

Compressed RDF The tool writes compressed RDF directly. A partial mark indicates compression available only through a bundled wrapper.

Public release A versioned package, executable, or container is available. A partial mark indicates source code only.

Table S1. Documented capabilities of VCF-to-RDF converters and related workflows, with evidence for each mark. ● Supported and documented. ◑ Partial or limited. ○ Not found in public documentation. Sources are the cited papers and the repositories, manuals, and registries named in each note.

	Representation			**Operation**			
Tool or workflow	RDF model	Header + genotype	Editable mappings	CLI	Reprod. workflow	Compr. RDF	Public release
Dedicated VCF-to-RDF converters							
VCF-RDFizer (this study)	●	●	◑	●	●	●	●
VCF-RDFizer uses the public VCF Core vocabulary to retain headers, records, alleles, and expanded or condensed sample data. Editable mappings cover only part of the output. RML (the RDF Mapping Language, whose declarative rules map source fields to triples) generates the record backbone, the nine triples per record that create each record and its fixed-datatype fields, or 5.3% of the HG005 graph. Emitters, converter code that writes triples directly rather than through mappings, generate samples, INFO, QUAL, and headers and are configured through options (Section S3.2). VCF-RDFizer uses a pinned Docker image and is released on PyPI, Docker Hub, and conda-forge. It produces HDT, a compressed binary RDF format, and COTTAS (Columnar Triple Table Storage), which stores triples in compressed Parquet tables; both can be queried without decompression [1, 2].							
TogoVar VCF2RDF [3]	◑	○	◑	●	●	○	●
TogoVar is a Rust command-line tool (`https://github.com/togovar/vcf2rdf`) with `generate`, `convert`, and `compress` commands. The documentation describes a very simple RDF model through a configuration template. For privacy, TogoVar converts only allele frequencies and omits sample and header data. YAML configuration selects the base IRI, namespaces, and INFO keys within fixed mapping rules. The `compress` command writes BGZF-compressed VCF, not RDF. A Docker image and release binaries are available.							

Table S1, continued

Tool or workflow	RDF model	Header + genotype	Editable mappings	CLI	Reprod. workflow	Compr. RDF	Public release
JVarkit VcfToRdf [4]	◑	◑	○	●	●	○	●
JVarkit runs as `java -jar jvarkit.jar vcf2rdf` (`https://jvarkit.readthedocs.io`). The project-specific `vcf:` namespace is documented only through examples. Output includes chromosomes, filters, samples, and genotypes (GT, DP, GQ, and PL), but omits INFO and FORMAT header descriptions. The documented default for `--hide` includes GT. The only options are `--hide` and `-o`. Releases are source tags without a published jar; Bioconda packages v2024.08.25.							
SPARQLing Genomics [5, 6]	◑	●	○	●	●	◑	●
The v0.99.11 manual documents `vcf2rdf`. The model uses FALDO, Dublin Core, PROV, and project-specific terms, with field predicates generated from the VCF header. The manual gives inconsistent namespaces. Every header-described field becomes triples, and per-sample calls carry INFO and FORMAT values. Only calls containing an ALT allele are emitted. Mappings are fixed in code. Compression is available only through the bundled `folder2rdf` wrapper's `--compress` option. A release tarball, Docker image, and Guix environment are available. The GitHub repository was archived in 2021 when development moved to GitLab.							
VCF2RDF (web) [7]	◑	◑	○	◑	◑	○	◑
VCF2RDF provides JavaScript for browsers and Node.js (`https://github.com/ibl/VCFr`). No command line is documented. Terms use `vcf2rdf.org/app/`, an offline host with no published schema. The paper describes header classes but not per-sample genotypes. Only source code is available, and the web service is offline.							
BioHackathon vcf2rdf [8, 9]	◑	◑	○	●	◑	○	◑
The BioHackathon 2013 tool provides a `vcf2rdf` executable (`https://github.com/fstrozzi/bioruby-vcf2rdf`). The Turtle output combines FALDO positions with an ad hoc allele namespace and includes per-sample genotypes and allele coverage. Header metadata is not mentioned. The README installs an unpublished Ruby gem, so only source code is available.							
General genomic-format tools							
BioInterchange [10, 11]	●	●	◑	●	●	○	●
The assessment covers BioInterchange 2 (v2.0.5, `https://github.com/BioInterchange/BioInterchangeC`). Version 1, conceived at BioHackathon 2012, does not read VCF. Version 2 converts VCF 4.2 to JSON-LD using the Genomic Feature and Variation Ontology (GFVO) [12]. Output contains one metadata object per file and per-sample genotypes on each feature. Python plug-in hooks and user annotations can change the output, but mapping files are not provided. JSON-LD tools generate RDF serializations, and compression was applied outside BioInterchange. A Homebrew package and installer are available.							
bio-vcf [13]	○	◑	●	●	●	○	●
The `bio-vcf --rdf` command and ERB templates such as `vcf2rdf.erb` define the output (`https://github.com/vcflib/bio-vcf`). Templates use ad hoc prefixes rather than a documented vocabulary. Header metadata and sample data are available to templates, but combining both requires user configuration. Releases are available as a Ruby gem and on Debian and Bioconda.							
ruby-rdf/rdf-vcf [14]	○	○	○	●	◑	○	●
The `vcf2rdf file.vcf.gz` command requires JRuby 9 and a tabix index (`https://github.com/ruby-rdf/rdf-vcf`). The tool has been unmaintained since the 0.1.2 gem release in September 2015. The documentation names no vocabulary and does not describe header, INFO, FORMAT, or genotype handling. Output is uncompressed N-Triples.							
Knowledge-graph construction pipelines							
VariantKG [15]	◑	●	○	◑	◑	○	◑
VariantKG annotates VCF with SnpEff and converts the annotated VCF with SPARQLing Genomics' `vcf2rdf`, inheriting the converter's header and genotype coverage. VariantKG extends the vocabulary with a project-specific ontology, including CADD scores, and links patient metadata through named graphs. No ontology file is published. A Gradio interface operates the workflow after manual installation of SnpEff, `vcf2rdf`, and Blazegraph. Only source code is available (`https://github.com/MU-Data-Science/GAF`).							
AgroLD VCF2RDF [16]	◑	○	○	○	○	○	◑
The component appears in [16] among the AgroLD conversion pipelines: GFF2RDF, GAF2RDF, and VCF2RDF (`https://github.com/SouthGreenPlatform/AgroLD_ETL`). AgroLD is ontology-based, but no VCF vocabulary, usage, or output is documented. Only source code is available.							
Virtual semantic access							
Semantic Beacons [17]	◑	○	●	○	◑	○	◑
Semantic Beacons maps Beacon API responses, rather than VCF files, to RDF on demand using RML over JSONPath. The RDF is served as Triple Pattern Fragments and queried with Comunica. The SO, FALDO, and GENO schema covers variant positions and alleles. The authors describe the schema as partial coverage of the Beacon specification. Beacon responses contain no header or per-sample genotypes. Implementation details are deferred to `https://github.com/SemanticBeacon/SemanticBeacon`.							

S2 Shared-input converter comparison

Table S1 records what each converter documents. The shared-input experiment compares four converters with VCF-RDFizer on the same two inputs, a 100,000-record HG005 slice and a 10,000-record, 16-sample slice of the 1000 Genomes chromosome-20 call set, using the same assessment measures (main Figure 6). The measures are agreement with the oracle for Q1–Q8, conversion wall time, peak resident memory, triples per record, and gzipped N-Triples size. The oracle is the set of expected answers computed directly from the source VCF with cyvcf2 and bcftools, and Q1–Q8 are the content questions of Table S5 (Section S4.1). Experiment 18 ran on `vcf-bench-1` on 8 October 2026 using `vcf-rdfizer-testing` commit `e3743590`. Each conversion had three replicates. The run records are archived in `benchmark-results/vcf-bench-1/18_converter_comparison/`. One Docker Compose file runs the complete experiment, so the host requires only Docker. The command is `CONV_DATA=... benchmarks/converters/run.sh`.

S2.1 Converters, builds, and options

Each converter used an upstream release image when available and the options that produced the most complete output, rather than necessarily the defaults (Table S2). Images built for the experiment used a digest-pinned Debian base and packages from `snapshot.debian.org` at a fixed date. Every downloaded artifact was checked against a pinned SHA-256. Run records retain the resolved Compose file and every image ID. Converter containers ran without network access and as the calling user. A static BusyBox `time -v`, mounted read-only in each container, measured wall time and peak resident set size. VCF-RDFizer reports the same quantities for each stage. VCF-RDFizer's wall time covers all stages, including container startup for each stage. Apache Jena `riot` 6.2.0 streamed each native output to N-Triples. The N-Triples were then counted, gzipped, and indexed with the VCF-RDFizer image's build of QLever, a SPARQL engine that answers queries from its own index of the graph [18].

Table S2. Converter versions, installation sources, and options used in the shared-input comparison.

Converter	Version	Obtained as	Options and output
VCF-RDFizer	v3.3.1 (`b25fb7b`)	Release image `sha256:3ad71b1a...2993`, driven by the release's command-line tool	Expanded samples, space-optimized storage, and N-Triples only. Output: gzip-framed N-Triples
JVarkit `vcf2rdf` [4]	Commit `fc2356a3`	Built from source with the git submodule. No jar is published, and the release tag moved after publication, so the commit pins the build. The SHA-256 of every fetched library jar is recorded	`--hide ''` (the default hides ALT, FILTER, and genotypes). Output: Turtle
TogoVar `vcf2rdf` [3]	v1.0.0-beta.12 (`f17835a6`)	Upstream Docker Hub image, by digest	`generate config` for the input assembly, followed by `convert --no-normalize` to retain POS, REF, and ALT as written. Output: Turtle
SPARQLing Genomics `vcf2rdf` [6]	0.99.11 (`1859f6f8`)	Upstream release image (a Guix pack), loaded from its release tarball after a checksum check	Defaults, which write INFO, FORMAT, and the header. Output: N-Triples
BioInterchange [11]	2.0.5 (`49d57012`)	The brewsci Homebrew bottle, available from the registry and identified by SHA-256. The source build no longer works because a dependency's repository was deleted	Defaults. Output: line-delimited JSON-LD. Context URLs return HTTP 404, so contexts from the pinned commit were inlined

The remaining tools in Table S1 were excluded for the following reasons. The VCF2RDF web service is offline and documents no command line. The BioHackathon `vcf2rdf` gem was never published. The `rdf-vcf` tool has been unmaintained since 2015 and names no vocabulary. The bio-vcf tool writes RDF only through user templates without a documented vocabulary. VariantKG uses SPARQLing Genomics' `vcf2rdf` for conversion. AgroLD documents no VCF conversion, and Semantic Beacons maps Beacon responses rather than VCF files.

S2.2 Shared validation protocol and query adaptation

The same comparison script evaluated every graph inside the VCF-RDFizer image. Every comparison used the image's validator, source-VCF oracle, QLever build, normalization procedure, and comparator. Only the SPARQL queries differed. Queries Q1–Q8 were adapted to each converter's vocabulary, while VCF-RDFizer used the queries supplied with the validator. When a graph lacked the required information, the question had no adapted query and was reported as *not represented*. Each missing query has a recorded reason supported by the converter's source code. The script refuses to run when a question has neither an adapted query nor a documented reason. Q9–Q13 were excluded because Q9 and Q10 count VCF Core predicates and classes, while Q11–Q13 hash VCF-RDFizer's record IRIs.

Adapted queries read only information present in the graph. When a converter represents different VCF values identically, the query interprets the shared representation according to the converter's source code. The comparison then measures the effect of the conflation on the tested inputs. JVarkit emits no FILTER triple for either PASS or '.', so the query reads an absent FILTER triple as PASS. JVarkit omits ploidy, so haploid calls are read as homozygous. JVarkit also leaves no-calls and records without GT untyped. TogoVar replaces CHROM with a configured reference IRI, which the Q1 query maps back using the same configuration. SPARQLing Genomics emits no record node. The queries therefore group calls by the record index encoded in call IRIs, as documented in the manual. SPARQLing Genomics also writes only the first ALT allele. BioInterchange writes a missing allele as REF, writes '*' as an empty string, and strips the `VCFv` prefix from `##fileformat`. TogoVar and SPARQLing Genomics lose the order of multiple FILTER codes.

S2.3 Results

Every conversion and comparison completed, giving 52 recorded steps with exit status 0. VCF-RDFizer matched the oracle for Q1–Q8 on both inputs. The remaining differences reflect the content of each converter's graph.

- **JVarkit.** Record IRIs use `urn:variant/CHROM:POS:REF`. Five pairs of 1000 Genomes records share CHROM, POS, and REF and therefore merge. For example, `20:283049 C>CGAT` and `C>T` both have ID '.'. Merging produced 9,995 records rather than 10,000, added five multiallelic records, and shifted every subsequent count. JVarkit matched the oracle for Q1–Q6 on HG005.

- **TogoVar.** The converter drops alleles outside `[ACGTURYKMSWBDHVN]`. The 13 symbolic records in the 1000 Genomes slice disappeared. In HG005, `chr1:1663386 C>*,T` became an SNV, affecting Q2 and Q3. The same five record pairs merged because TogoVar emits no record node. Sample data and header lines are omitted by design.

- **SPARQLing Genomics.** The converter emits only calls containing an alternate allele. The 1000 Genomes graph therefore retained 1,401 of 10,000 records and omitted homozygous-reference calls, so the allele numbers that Q6 recomputed counted carriers only. Multiallelic records were recognized only when a call included an allele index of 2 or more, representing

19 of 70 such records. Every HG005 call contains an alternate allele, and Q1–Q7 matched the oracle on HG005. For Q8, the graph listed 10 FILTER header lines rather than 9, because htslib, the library through which the converter reads the VCF, adds a PASS declaration (`##FILTER=<ID=PASS>`) that the source header lacks.

- **BioInterchange.** The HG005 header document, line 2 of 100,003, was invalid JSON because the converter did not escape strings. The resulting graph lacked the header information required for Q7 and Q8. All 100,000 record documents (lines 3–100,002) parsed, as did the context and summary documents (lines 1 and 100,003). BioInterchange matched the oracle for all eight questions on the 1000 Genomes slice.

Table S3. Conversion cost and output size on shared inputs. Wall time is the median of three replicates, and memory is the largest peak resident set size. Output size is measured for replicate 1 after conversion to N-Triples. QLever indexing times refer to the same graphs. N-Triples.gz ÷ VCF.gz divides each graph, re-serialized by `riot` and compressed as one gzip stream (`gzip -6`), by the BGZF copy of the input that every converter read (HG005: 89.1 and 3.50 MB); VCF-RDFizer's own gzip-framed output (75.8 MB) is 24.7 times the original 3.07 MB gzip input (main Figure 7b).

Converter	Input	Wall (s)	Peak RSS (GB)	Triples	Triples per record	N-Triples.gz ÷ VCF.gz	QLever index (s)
VCF-RDFizer	HG005	91.6	1.85	17,098,746	171.0	25.5	20.8
	1000 Genomes	31.4	0.93	5,359,578	536.0	136.9	7.0
JVarkit	HG005	2.2	0.43	1,401,613	14.0	2.1	1.8
	1000 Genomes	1.4	0.30	1,020,454	102.0	18.2	1.4
TogoVar	HG005	1.3	0.01	5,728,120	57.3	15.4	6.8
	1000 Genomes	0.1	0.01	564,940	56.5	35.0	1.0
SPARQLing Genomics	HG005	6.3	0.01	2,958,769	29.6	4.8	3.3
	1000 Genomes	0.4	0.01	180,053	18.0	4.1	0.5
BioInterchange	HG005	1.5	0.02	4,559,438	45.6	17.0	6.6
	1000 Genomes	0.4	0.01	2,031,215	203.1	154.3	3.2

Replicates. For each converter, the three outputs were byte-identical apart from two exceptions. VCF-RDFizer's gzip framing differed, but sorted triples were identical across all three replicates of both inputs. For HG005, the sorted triples also matched the v3.1.0 output (Table S13). BioInterchange outputs differed only in the `Context` and `Summary` documents, which record run paths and timestamps.

IRI warnings. The `riot` parser accepted every output but warned about IRIs from three converters. VCF-RDFizer's `file://<name>.vcf#record/n` IRIs place the file name in the URL host position. The parser classified the IRIs as "unwise" and issued 23.6 million warnings for HG005. SPARQLing Genomics' `origin://<md5>@<k>` IRIs use deprecated user information and produced 3.1 million warnings. JVarkit's `urn:variant/...` IRIs have an invalid URN namespace and produced 0.6 million warnings. No warning prevented parsing or indexing. TogoVar and BioInterchange outputs produced no warnings.

S3 Representation and implementation details

S3.1 Model and sample profiles

VCF-RDFizer uses the VCF Core vocabulary (`vcfc:`, `https://w3id.org/vcf-core/vocab#`) to represent file and header assertions, variant records, and sample/genotype information [19]. The model aligns selected terms with FALDO, the Sequence Ontology, HERO, and GA4GH

VRS [20, 21, 22, 23]. Value and cardinality constraints are declared as SHACL shapes (the W3C Shapes Constraint Language, which checks a graph against declared rules [24]), not as OWL restrictions in the vocabulary. Figure S1 shows the emitted backbone, and Table S4 separates implemented support from completed assessment.

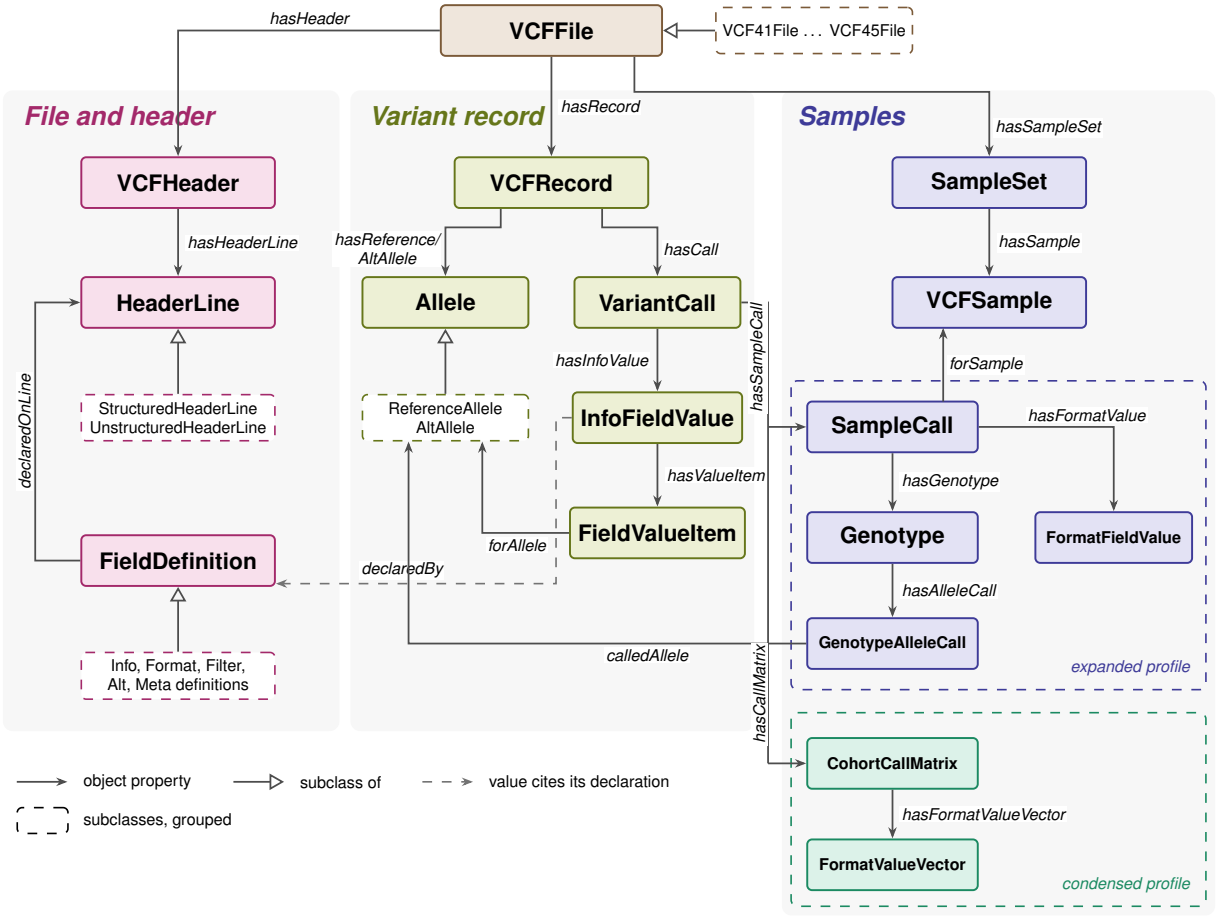


Figure S1. Simplified VCF Core model emitted by the evaluated converter. A versioned file connects header declarations, records, and sample information. Values retain links to the context needed for interpretation. Expanded output exposes sample calls, genotypes, and allele calls, whereas condensed output uses sample-ordered vectors. The complete vocabulary is documented at `https://ecrum19.github.io/vcf-core-vocabulary/`.

VCF stores sample-level FORMAT values in a table, with each record containing values for every sample and FORMAT field. The expanded profile represents individual sample calls and FORMAT values as directly addressable RDF resources. This allows consumers to query individual values without decoding a vector, making the expanded profile suitable for single-sample and smaller multi-sample files. For larger cohorts, however, the repeated structure increases graph size, memory use, storage requirements, and indexing costs with VSF (Equation 1). For V records, S samples, and F FORMAT keys in a uniform layout, the sample-layer structure scales as

$$\begin{aligned} T_{\text{expanded}} &= \mathcal{O}(S + VSF), \\ T_{\text{condensed}} &= \mathcal{O}(S + VF). \end{aligned} \tag{1}$$

The condensed profile reduces this overhead by declaring samples once and storing FORMAT values in sample-ordered vectors. The trade-off is that individual values require decoding rather than direct graph access. Both profiles preserve the same VCF information but offer different balances between direct access and compactness. Each converted file identifies its profile through `vcfc:representationProfile`. The VCF-RDFizer sample-representation guide provides detailed

examples, scaling calculations, and guidance for choosing a profile (`https://github.com/ecrum19/VCF-RDFizer/blob/v3.3.1/docs/sample-representation-guide.md`).

Deterministic observation IRIs incorporate the source file and record position. Separate linksets add cross-file variant identity. A linkset is an N-Triples file, kept apart from the converted graph, that holds the links written by *linkers*, declared rules that connect VCF records or their alleles to external identifiers (Section S5.1). GA4GH VRS computed identifiers could complement the evaluated SPDI route (NCBI's Sequence:Position:Deletion:Insertion variant notation [25]) but were not implemented [23].

Table S4. Implemented capabilities and assessed scope. Each status reports release support followed by the completed evaluation in italics. Section references without an S prefix refer to the main text.

Capability	Status	Notes and limits
Core fields (CHROM, POS, ID, REF, ALT, FILTER)	Supported (v3.1.0) *Validated*	Q1–Q4 equal in completed comparisons and in the consumer WGS validation run (Section S4.2)
QUAL	Supported *Q11 record digest*	Every record has a typed literal. Missing QUAL is `vcfc:Null`
INFO	Supported, structured by default *Q11, Q12 digests*	Typed value per record and key, per-allele items for Number=A/R/G, plus the complete `infoRaw` string
FORMAT, sample, and genotype values	Supported *Q5–Q6 exact, Q13 digest*	Conversion preserves half-calls (for example, `./1`), haploid calls, and triploid and tetraploid calls (Table S15). Detecting injected faults in called alleles, sample indexes, and phasing requires the deeper profiles
Header metadata	Supported *Census queries and shapes*	Header attribute–value agreement requires the deeper profiles, not merely the default profile
Multi-sample conversion	Supported *Measured, 1–2,504 samples*	Condensed adds 5 triples per sample, while expanded adds 25 per sample per record (627M triples for 10,000 records at 2,504 samples). Neither profile was applied to the complete 1000 Genomes chromosome-20 call set, only to 10,000-record slices of it (Section 3.3)
Structural and symbolic alleles	Lexically supported *HGSVC2 converted, Q2 passes on symbolic and breakend fixtures*	ALT retains the source text. Only the deeper profiles detect allele-value faults
Queryable HDT, COTTAS	Supported *Measured*	All 118 HDT and 34 COTTAS artifacts passed decode checks. COTTAS was smallest throughout (Section 3.3)
Gzip/Brotli packaging	Supported *Covering set (Section S10)*	The covering set is 10 configurations that together include every option value and every allowed pair of values. Querying requires unpacking. No codec comparison was performed
Policy annotations	Plug-in v0.2.0, shipped since v3.3.0 *Real VCFs, up to a complete single-sample VCF*	Rules in ODRL, the W3C policy language for permissions and prohibitions [26], target files and declared graph selections and name purposes from DUO, the GA4GH Data Use Ontology [27]. Every view was checked against a VCF-side oracle. Evaluation used simulated consents and single-sample files. The plug-in provides governed release, not anonymization (Section 3.2)
External linking	SPDI, Ensembl-gene, and MyVariant linkers (v3.3.0) *Real VCFs*	SPDI joins eligible alleles under the stated reference and normalization assumptions. The live MyVariant.info linker (Tier 3) confirmed 92–93% of rsID rewrites (Section S5)

S3.2 Conversion and incremental representation construction

Main Figure 2 summarizes the conversion pipeline. Conversion derives record, header-line, and file-metadata TSV sources because the implemented RML path does not read VCF directly. RMLStreamer applies Turtle-encoded mappings to the record backbone, while direct emitters add samples and values requiring row-specific datatype or structure handling [28, 29]. RML contributes nine triples per record, or 5.3% of the HG005 graph, and emitters contribute the remaining 94.7%. The percentages describe the evaluated hybrid implementation rather than a general limitation of RML.

Each output part, the triples written by one conversion stage, is appended either to a plain N-Triples file or, as a separate gzip member, to a concatenated-gzip stream. The part is deleted after appending. Standard gzip readers expose the members as one N-Triples sequence. HDT and COTTAS construction, RDF compression, and transport packaging are independent options. The HDT and COTTAS builders read the graph one chunk of complete N-Triples lines at a time, so gzip-framed output is never expanded into a full uncompressed copy. HDT parts are merged pairwise with `hdtc`, while COTTAS parts are merged in one streaming k-way pass [30, 1, 31, 32, 33]. Default chunks target 512 MiB, with bounds of 128 MiB and 1 GiB. Six feasibility runs used smaller chunks. Before raw RDF is removed, each artifact is decoded with the corresponding native reader. The decoded triple count is compared with the source graph. The check covers decoding and count preservation, not complete RDF-term equality. QLever decodes an input artifact and builds its own index, so QLever timings do not measure native HDT or COTTAS querying. The native COTTAS route uses a DuckDB-backed Comunica engine over Parquet [34].

S4 Validation specification, scope, and sensitivity

S4.1 Queries, normalization, and engines

Validation compares 13 SPARQL results with answers derived from the source VCF using cyvcf2 and exact FILTER text from `bcftools query` [35, 36, 37]. Fourteen preflight queries first inspect syntax-independent graph properties, including missing values, POS typing, core-property cardinality, sample representation, inventories, and triple counts. Table S5 defines the source-comparison questions. Each query summarizes one part of the VCF (main Table 1) as counts, histograms, or hash-bucket digests rather than as full record lists, so its result stays small on graphs of any size and its expected answer can be recomputed from the VCF text without reading RDF. The queries use only standard SPARQL 1.1, including `SHA256` for the digests and no order-dependent `GROUP_CONCAT`, so the same query text runs on every engine. Queries and oracle were developed against one declarative test fixture from which the test VCF, its expected graph, and its expected oracle summary are all derived. Its records exercise every query branch: a transition and a transversion SNV, a deletion, a multiallelic and a no-ALT site, PASS and failed FILTER values, and missing and non-diploid genotypes. A continuous-integration test checks that the oracle computed from that VCF equals the hand-derived summary, and the queries also run in process under rdflib, an independent SPARQL implementation. A container test then runs every query on Comunica, QLever, and the native HDT and COTTAS engines and requires identical answers, and fault injection (Section S4.3) measures which corruptions the queries miss. Main Figure 3 shows Q3 next to its oracle, and the results site shows every query and its oracle.

Normalization parses count fields as integers and sorts rows by their key fields before comparison. Normalized results retain contig and sample identifiers, distinguish `PASS` from `.`, and report missing, extra, or differing rows. Q5–Q6 apply only to inputs with sample data and `GT`. Q11–Q13 hash each record, INFO value, or FORMAT value together with its own IRI using SHA-256, place it in one of 256 buckets according to the first byte of the hash, and compare the per-bucket counts from the graph and the VCF. The digest summaries extend value coverage: a changed value usu-

Table S5. The 13 source-comparison queries. A comparison passes when the normalized SPARQL and parser-derived results agree exactly. Q4 uses exact bcftools FILTER text. The Ti/Tv ratio and allele frequency are derived from the Q3 and Q6 counts, respectively, after those counts agree. Q11–Q13 compare hash-bucket summaries. The expected results for Q9, Q10, Q12, and Q13 depend on the selected output options, such as the sample profile and INFO mode.

	Query	Result compared, fields, and scope
Record level: every input		
Q1	Record density	Records per contig and 1-Mb window (`CHROM`, `POS`), checking coordinates and integer positions
Q2	Allele shapes	Records per class: SNV, multiallelic, insertion, deletion, substitution, symbolic/breakend, no ALT (`REF`, `ALT`)
Q3	Ti/Tv	Transitions, transversions, and biallelic SNVs with single-base A/C/G/T alleles
Q4	FILTER values	Records per exact FILTER string: `PASS`, `.` (not applied), or failed codes, combined codes kept whole
Genotype level: inputs with sample columns and `GT`		
Q5	Genotypes	Calls per sample and genotype class (no GT, missing, haploid, homozygous, heterozygous, other ploidy); phased and unphased calls are pooled
Q6	Allele counts	Sites per recomputed (AC, AN) pair for single-ALT sites with complete 0/1 haploid or diploid calls
Metadata and graph structure		
Q7	File metadata	Exact `fileformat`, `reference`, and `source` strings, with markers for absent declarations
Q8	Header census	`##` lines per header key (INFO, FORMAT, FILTER, contig, ...)
Q9	Predicate census	Triples per predicate against the counts expected from the VCF
Q10	Class census	Resources per `rdf:type` against the expected inventory
Value preservation: IRI-bound digests		
Q11	Record digest	Records per bucket over each record's IRI, `CHROM`, `POS`, `ID`, `REF`, `ALT`, `QUAL`, `FILTER`, and raw INFO
Q12	INFO digest	Structured INFO values per bucket, bound to record and key. Empty in raw-only INFO mode
Q13	FORMAT digest	FORMAT values, including `GT` (expanded), or sample-ordered vectors (condensed) per bucket

ally moves its item to another bucket, but changes that preserve the bucket histogram, such as two that offset each other, can go unnoticed.

The same SPARQL text runs under QLever, Comunica over N-Triples, Comunica over HDT, and the DuckDB-backed COTTAS engine [18, 38, 39, 40, 34]. The default SHACL profile (`core`) checks cardinality and datatypes, while two deeper profiles check that related values agree (`consistency`, for example GT text and called alleles) and that indices are unique and records ordered (`vocabulary-sparql`); all three together form the `full` set. A passing query or shape result establishes only the specified summary or enabled constraint.

S4.2 Completed comparisons and the consumer WGS validation run

All 78 validation runs in the core benchmark completed. The comparisons yielded 1,006 exact matches and eight genotype comparisons classified as not applicable. The zero-record and sites-only fixtures each account for four not-applicable comparisons because neither fixture contains genotypes. The two sites-only validations were rerun after the malformed fixture prevented the original checks from completing (Section S8). The largest graph checked by paired validation, which compares SPARQL answers with answers derived from the VCF, had 17.1 million triples (100,000 HG005 records). The default shapes found no violations in 60 validations on graphs up to 0.96 million triples. Repeatability checks produced identical sorted triples, preserved digests through packaging, and left artifacts unchanged after re-indexing.

The complete HG005 retrieval run compared all 13 answers at 657,425,805 triples without

Table S6. Validation results by assessment layer. Counts report completed checks. Fault sensitivity applies only to the defined fault set and enabled profiles.

Check	Passed	Evidence provided
Is the graph faithful to the source VCF?		
N-Triples syntax (`rapper`)	78/78	The serialization parses
Paired queries vs cyvcf2 (Q1–Q13)	1,006/1,006[†]	Exact equality of normalized query results
Cross-check invariants	1,160/1,160	E.g. transitions + transversions = biallelic SNVs
Cross-engine agreement	25/25	All four engines agreed. The other 53 completed validations used one engine
Native decode and triple count	152/152	118 HDT and 34 COTTAS artifacts. Source = decoded count
Is the pipeline reproducible?		
Determinism, two runs	1/1	Identical SHA-256, 17,098,746 triples, and 0 duplicate lines
Compress and decompress	1/1	Lossless through the packaging path
Index idempotence	1/1	Re-indexing does not perturb the artifact
Would a fault actually be noticed?		
Faults caught, queries only	96/113 (0.850)	17 faults, spanning 10 fault classes, are invisible to the queries
with the default profile	96/113 (0.850)	Cardinality and datatypes catch none of the 17
with all three profiles	**113/113** (1.000)	Uniqueness and value agreement detect all 17. Both controls pass
SHACL shapes, default profile	60/60	Cardinality and datatype constraints, 0 violations

[†]Eight further comparisons, Q5 and Q6 on the zero-record and sites-only fixtures, were verified as not applicable and are excluded from the 1,006 applicable comparisons.

SHACL and is not included in the 1,006-comparison denominator. The consumer WGS validation run converted 250,000 records of `NG131FQA1I`, a single-sample consumer whole-genome sequencing VCF from Dante Labs donated through the Personal Genome Project (Table S14), into 58,231,176 triples. The initial v3.1.0 run reported three discrepancies. The oracle omitted emitted phase sets and compared non-canonical decimal QUAL strings. Direct field comparison found no conversion differences. Canonicalizing QUAL reproduced the graph-side digest. Version 3.3.1 corrected the oracle, added expectations for additional emitted structures, and validated the default shapes in batches of whole records. These shapes are record-local: each constraint judges one resource from its neighboring triples, and each batch holds whole records plus the file-level triples they reference, so batching does not change the verdict. The published v3.3.1 image matched all 13 expected results and reported no violations across 117 batches. The shapes issued 46 non-blocking recommendations for missing VCF 4.5 `Source` and `Version` attributes. The batched run took 91 minutes with four workers, and available memory never fell below 19.5 GB.

S4.3 Fault sensitivity and oracle limits

The fault set applies 113 targeted changes, one at a time, to small, correct expanded and condensed graphs of a fixture VCF; the changes alter coordinates, alleles, filters, metadata, phasing, sample indexes, and relationships. Source-comparison queries detected 96 faults. The default shapes detected none of the remaining 17 faults. All three profiles together detected every fault and accepted both controls, the uncorrupted graph in expanded and in condensed form. The 17 additional detections span 10 fault classes: seven injected in both sample profiles (allele kind, allele value, per-allele INFO item links, record index, contig count, header attribute values, and field-declaration ownership) and three injected only in the expanded profile (called-allele links, sample index, and phasing status). The phasing fault changed one genotype's `vcfc:phasingStatus` from phased to unphased while leaving its GT text unchanged, so neither Q5, which classifies the GT text after mapping `|` to `/`, nor the Q13 digest of that text could detect it. The deeper profiles took

33.7 and 73.2 seconds on approximately 2,000 triples, compared with 1.7 seconds for the default profile. The detection rate applies to the defined fault set on a small graph, not to arbitrary data.

Version 3.3.1 applies record-local default shapes in batches at any graph size. The deeper profiles are cross-record: they compare records with each other, so the whole graph must be loaded into memory, and a graph-size limit still applies. The parser and graph routes read different representations but share selected definitions and representation expectations. An incorrect dependency can therefore affect both the converter and its oracle, as occurred for the dangling allele-reference defect (Table S16), in which the census expectation shared the emitter's rule. Text comparisons, source-derived summaries, structural constraints, and fault injection provide complementary evidence rather than an infallible oracle. Local-allele semantics remain outside the completed oracle coverage.

S5 Linked genomic workflow: protocol, policies, and detailed results

S5.1 Scope, linking, and policy evaluation

The linked workflow tests relationships shared by the three motivating applications rather than implementing complete clinical or governance systems. For reanalysis, the workflow joins observations to fixed ClinVar and population-frequency releases. For pharmacogenomics, the workflow tests cross-resource linking but not diplotype interpretation, medication orders, or prescribing guidance. For consent-aware reuse, the workflow evaluates simulated purposes, withdrawals, and record selections but not real consent adjudication or authenticated access. Figure S2 summarizes the implemented path.

Linking. `vcf-rdfizer-link` executes Turtle-declared linkers and writes a separate N-Triples linkset with coverage and skipped-record reports. Linkers belong to three tiers. Tier 1 rewrites identifiers already in the VCF without checking them, as the `rsid-dbsnp` linker does when it turns rsIDs into dbSNP IRIs. Tier 2 computes links against reference data pinned by checksum and genome assembly, as the SPDI and Ensembl-gene linkers do. Tier 3 confirms identifiers with a live web service, sending rate-limited requests and caching the responses. The evaluated SPDI linker [25] harmonizes contigs to RefSeq accessions and minimally trims each explicit allele, removing the leading and trailing bases that REF and ALT share. Explicit alleles are ALT alleles written as bases, not symbolic alleles such as `<DEL>`, breakends, or the spanning deletion `*`. The gene linker records Ensembl 116 genes overlapped by the REF span. Minimal trimming supports the tested joins but does not establish canonical identity in every repeat context. The live MyVariant.info linker confirms rsIDs [41]. Service failure stops the linker without publishing a partial linkset.

Policy evaluation. `vcf-rdfizer-policy` evaluates ODRL rules over files or declared graph selections [26]. The VCF Core policy profile applies default-deny semantics, so anything not explicitly permitted is withheld, and deny-overrides semantics, so a prohibition wins over a conflicting permission. By default, it uses four DUO purposes: general research use, health/medical/biomedical research, disease-specific research, and clinical care [27]. When a source resource is withheld, ownership rules also remove the calls, alleles, sample calls, and headers that depend on it. Each release view is checked in four ways: its recorded policy digest must match the policy; it must contain nothing that an applicable prohibition covers and nothing that no applicable permission covers; none of its triples may point to a source resource missing from the view; and its records must equal those released when the same policy is applied to a minimal graph built directly from the VCF text, bypassing the converter. Released genotypes

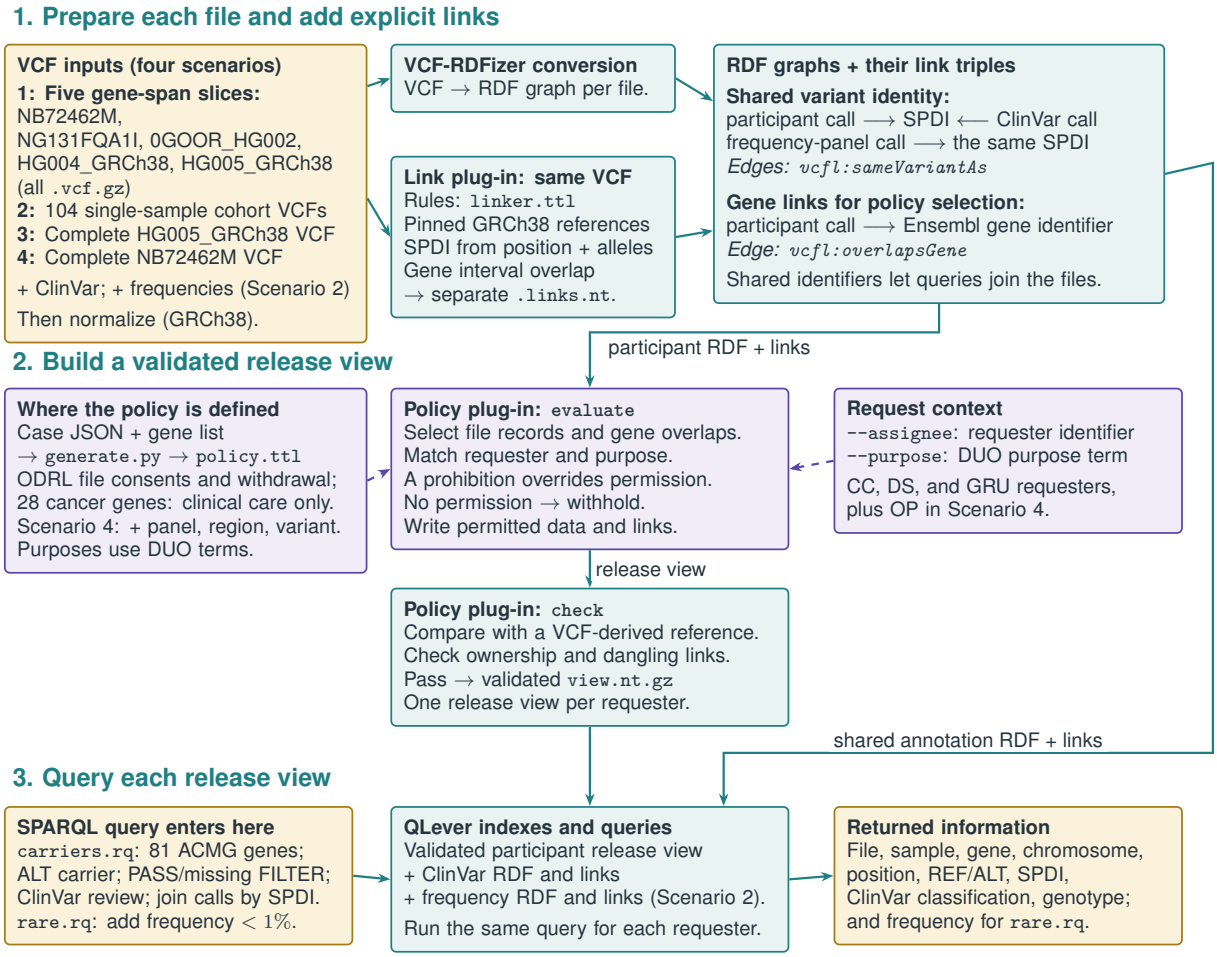


Figure S2. The implemented path from VCF files to a permitted answer. The diagram follows the use-case runner, policy generator, linker definitions, and SPARQL files. Linkers read normalized VCFs directly, and links are stored separately from conversion output. Inputs are restricted to the ACMG gene spans except in Scenarios 3 and 4. Policies determine each materialized release view from the supplied requester identity and purpose. The SPARQL query defines the analysis filters. The cancer-gene policy uses Ensembl overlap links, whereas the query reports the ACMG genes named in ClinVar's annotation. Requesters are clinical care (CC), disease-specific cardiovascular research (DS), general research use (GRU), and, in Scenario 4, care by the participant's own physician (OP).

retain source IRIs and are not anonymized. The plug-in records obligations (ODRL duties, such as deleting released data after use) but does not enforce them. Selector parameters are placed inside the outer SPARQL group. An earlier trailing placement produced an empty region selection and was corrected before the reported experiments.

S5.2 Application question, comparison workflows, and evaluation scenarios

The application identifies observed variants with a ClinVar classification in the fixed ACMG SF v3.2 81-gene set [42]. The application also identifies the participant and gene associated with each match and the matches each requester may receive. ClinVar records require at least one review star. Pathogenic and likely pathogenic matches are reported separately but are not treated as reportable secondary findings.

The RDF workflow links converted VCFs to ClinVar and gene resources, materializes each requester view, and queries the view with SPARQL against QLever [18]. The conventional workflow harmonizes contigs, annotates `CLNSIG`, `CLNREVSTAT`, and `GENEINFO` with bcftools, and applies the same case definition and release rules in Python [36]. The comparison requires exact agreement

between the workflows' sorted tuples of participant, gene, contig, position, and alleles. The workflows share preprocessing and the case definition but use separate downstream implementations.

Scenario 1 contains ACMG gene-span slices from five single-sample VCFs: two consumer VCFs donated to the Personal Genome Project (`NB72462M` and `NG131FQA1I`) and three VCFs of GIAB samples (the precisionFDA HG002 file and the HG004 and HG005 benchmark VCFs). Scenario 2 contains 104 high-coverage 1000 Genomes participants. Scenarios 3 and 4 contain the complete HG005 and NB72462M VCFs, respectively [43, 44, 45]. Scenarios 1 and 2 use only records inside the 81 ACMG gene spans from Ensembl 116 (7.1 Mb in total), after normalization and splitting of multiallelic records. Scenario 2 removes repeated panel-level INFO and stores the required frequencies once in a sites-only graph. This graph retains AF and the five population-specific AF fields; AC and AN are omitted because sample subsetting recalculated them for the cohort rather than the reference panel. The rare-variant query extends the participant–ClinVar SPDI join with the frequency graph and requires panel AF < 0.01; the conventional workflow applies the same threshold using a position-and-allele lookup. All scenarios use the 2026-09-13 ClinVar release [46]. Simulated release rules combine file permissions, four seeded withdrawals in Scenario 2, a 28-gene cancer restriction, and requester purposes for clinical care, cardiovascular research, and general research. The simulated consent for HG005 permits only clinical care, so the Scenario 3 views for DS and GRU are empty. Scenario 4 adds care by the participant's physician. The policies combine a 40-gene cardiovascular panel, an APOE region, and one DSP variant. In Scenario 4, every requester receives a non-empty subset of the same complete VCF. No scenario tests selective masking of samples within a multi-sample VCF.

Four applied changes test a withdrawal, a new purpose, replacement of the cancer panel, and an HFE C282Y restriction. The comparison tests whether both workflows release the same fixture records after each change. Line counts describe the two implementations and do not measure usability or maintainability.

S5.3 Linking results, matches, and rule changes

The SPDI linker emitted links for eligible explicit alleles under the stated assembly and normalization assumptions. A seven-variant BRCA1 check agreed with NCBI expressions outside repeats, while repeat-region spans differed from NCBI contextual representations. The Tier 3 MyVariant.info linker confirmed 9,223 of the 9,951 Tier 1 rsID rewrites in `NB72462M` and 9,479 of 10,316 in `NG131FQA1I`. The 21 batch requests were sent to MyVariant.info during one run on 2026-09-28. Version 3.3.1 confirmed the same links offline using the recorded responses, keyed by request (Section S6.2). The testing repository publishes the responses. Reproducing the result requires the recorded responses rather than the live service, where data can change. The Ensembl live variation service returned HTTP 500 after three retries, and no partial linkset was published.

The RDF and conventional workflows returned identical requester-specific match sets in every scenario. Every release view passed the policy and source checks. The complete HG005 query matched the gene-span query result. The Scenario 2 frequency join agreed with the conventional position-and-allele lookup (Figure S3).

The workflows also agreed on released-record counts in Scenarios 1, 3, and 4. Scenario 2 differed, as described below. In Scenario 4, the participant's physician received the complete VCF, while clinical care excluded six APOE records. Disease-specific research retained only the cardiovascular panel. General research excluded the cancer-gene, APOE, and single-variant selections.

The record comparison exposed gene-linking gaps that match comparison and release validation did not detect. Version 3.3.0 skipped spanning-deletion records, leaving the records outside gene-based selections. Version 3.3.1 links spanning deletions by REF span, and Scenarios 1, 3, and 4 agree with the conventional workflow record for record. Symbolic alleles and breakends remain outside the gene linker because determining variant extent requires additional interpre-

Table S7. Calls linked by each linker, summed over each scenario's files. SPDI eligibility requires explicit alleles. Every scenario used v3.3.1. The gene linker also links spanning-deletion (*) records omitted by SPDI: 65 in Scenario 1 and 15,032 in Scenario 4. A further 19 HG002 calls in Scenario 1 overlap genes but are ineligible for SPDI linking. The gene linker excludes symbolic alleles and breakends.

		Scenario					
Linker	**Tier**	**1**	**2**	**3**	**4**	**ClinVar**	**Result**
`spdi`	2	51,302	1,149,198	3,887,200	4,991,223	331,413	Links every eligible call and joins VCFs to ClinVar across contig names. Complete HG005 VCF: 89 s
`ensembl-genes-grch38`	2	51,386	1,149,198	1,704,954	2,063,627	—	Reloads the 108 MB GFF3 per file: 19 of Scenario 2's 22 min
`rsid-dbsnp`	1	20,267	—	—	4,974,526	—	PGP VCFs' rsIDs, rewritten unchecked
`rsid-myvariant`	3	18,702	—	—	—	—	92–93% of tier-1 links confirmed in 21 requests. The three inspected unconfirmed rsIDs were merged
`rsid-ensembl`	3	0	—	—	—	—	Persistent HTTP 500. Failed closed without publishing a partial linkset

Table S8. Requester-specific participant–variant–gene matches. The RDF and conventional workflows agree in every cell. All indicates no policy, and OP indicates care by the participant's own physician in Scenario 4. CC, DS, and GRU indicate clinical care, cardiovascular disease-specific research, and general research use, respectively. Triple counts exclude links. P/LP rows count the matches whose ClinVar classification is pathogenic, likely pathogenic, or pathogenic/likely pathogenic. The rare-variant row also joins panel frequencies at AF < 0.01.

				Matches by requester				
Scenario	**Files**	**Records**	**Triples**	**All**	**OP**	**CC**	**DS**	**GRU**
1: five gene-span slices	5	51,386	9.0M	7,211	—	2,878	2,988	1,987
P/LP				0	—	0	0	0
2: cohort	104	1,150,288	67.8M	173,102	—	63,529	92,870	48,207
P/LP				8	—	4	6	4
rare in the panel				4,154	—	1,621	2,198	1,144
3: complete HG005	1	3,887,810	662.4M	1,496	—	1,496	0	0
P/LP				0	—	0	0	0
4: complete NB72462M	1	5,063,417	1,209.7M	1,382	1,382	1,382	722	983
P/LP				0	0	0	0	0

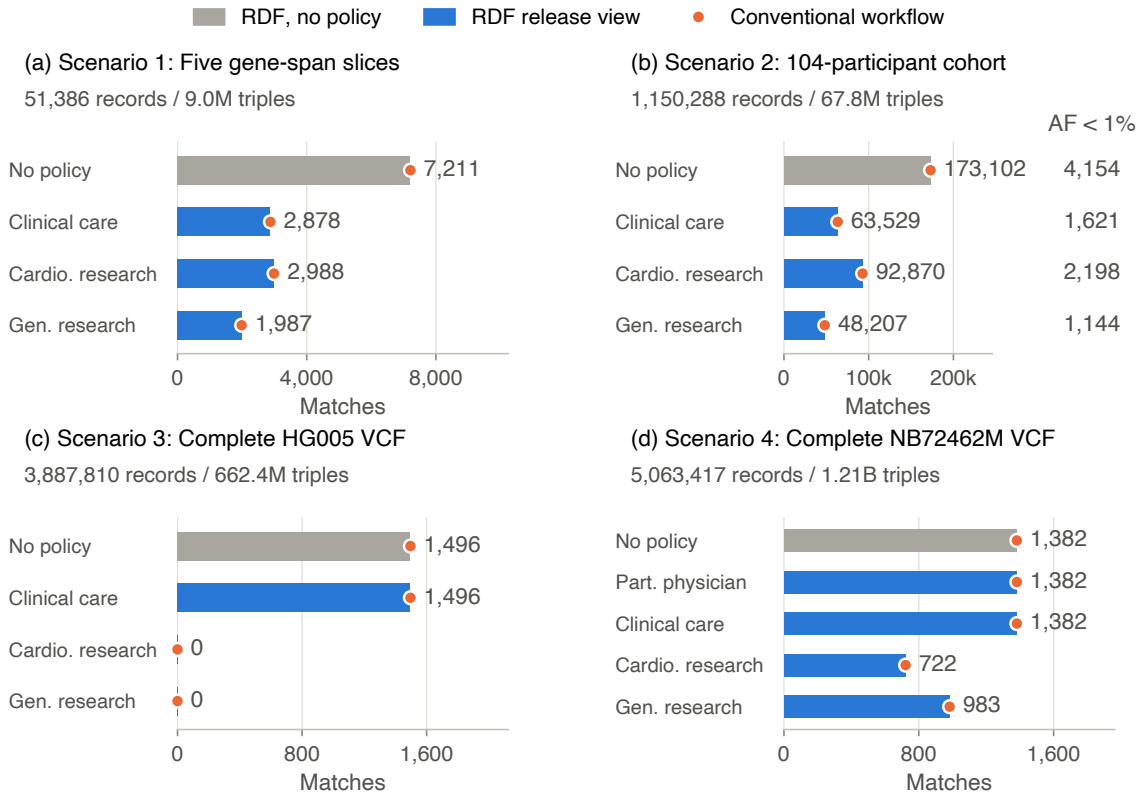


Figure S3. All four scenarios returned the same match sets in both workflows. Results are from v3.3.1; bars show RDF counts and dots show conventional counts, with exact agreement of the underlying participant–variant–gene tuples. Each panel has its own horizontal scale. No policy denotes the unrestricted query. Records and triples describe the participant input graphs before linking, excluding ClinVar and the frequency graph. The Scenario 2 AF column gives the frequency-filtered subset (AF $<$ 0.01) for each requester, also identical in both workflows. In Scenario 3, HG005's simulated consent permits only clinical care, leaving both research views empty. Match agreement does not establish correct release: Scenario 2's RDF views contained 110 extra records for cardiovascular research and 58 for general research, outside the explicit-allele case definition. Table S8 also reports pathogenic and likely pathogenic subsets; Table S11 gives stage costs.

Table S9. Scenario 4 records withheld by each rule and requester from the complete NB72462M VCF (5,063,417 records after normalization). The simulated consent permits clinical care and general research use. Care by the participant's own physician is declared as a narrower purpose than clinical care, and DUO treats disease-specific research as narrower than general research use, so the physician and the cardiovascular research requester also fall within the consent. A record withheld by several rules is counted once under the first rule. For DS, APOE and the DSP variant fall outside the cardiovascular panel. OP denotes the participant's own physician. Other abbreviations follow Table S8.

Rule	Selects records by	OP	CC	DS	GRU
Cancer genes (28): clinical care only	gene links	—	—	2,513	2,513
Cardiovascular panel (40): DS sees only this	complement of the panel's gene links	—	—	5,054,507	—
APOE: own physician only	coordinates on `chr19`	—	6	—	6
DSP variant: withheld from GRU	SPDI identity	—	—	—	1
Records released		5,063,417	5,063,411	6,397	5,060,897
Matches		1,382	1,382	722	983

tation. In Scenario 2, the RDF workflow released 110 records to DS and 58 to GRU that the conventional workflow withheld. Every additional record represents a symbolic structural variant (`<INS:ME:...>`, `<DUP>`, or `<INS>`) in a restricted cancer-predisposition gene, such as *BRCA2*, *NF2*, or *STK11*. Match sets remained unchanged because the case definition joins only explicit alleles. Every view passed the release checks because the checks use the same gene links as the policy evaluator.

The HFE C282Y restriction required a policy edit without changes to the RDF policy engine. Because the rule targets the variant's SPDI identity, it also selected the HG002 record, although that file writes the contig as `6` rather than `chr6`. The conventional workflow required a small code and data edit. The other three changes were similarly sized data edits in both workflows.

Table S10. Authored content and rule changes in Scenario 1, measured in non-blank, non-comment lines. Both workflows returned the same requester-specific records after every change. POA denotes DUO's population-origins-or-ancestry purpose, which falls outside the evaluated GRU permission.

	RDF workflow	**Conventional workflow**
Initial authoring		
Rules written	72: policy 41, query 31	131: shell 33, Python 98
Data written	28 gene IRIs and the query gene list (1 line)	Consent and requester tables (100 lines) and gene list (81)
Linkers, selectors, and profile	Shipped with the tool	—
Rule changes		
A participant withdraws	Policy $+2/-1$	Data $+2/-1$
A requester with a new purpose (POA)	Purpose vocabulary $+2$	Data $+4$
The panel becomes 40 cardiac genes	Policy $+40/-28$ (gene IRIs)	Data $+41/-29$ (gene symbols)
One variant (HFE C282Y), clinical care only	Policy $+6$, no engine change	Code $+3$, data $+9/-1$

S5.4 Stage costs

Conversion and linking run once per scenario. View materialization, validation, indexing, and querying run separately for each requester. The conventional workflow runs once per scenario for all requesters: it harmonized contigs, annotated with ClinVar, selected records, and applied the release rules in 4–91 seconds, against 5.7–105 minutes for the RDF workflow's conversion and linking alone. Queries over non-empty views took 171–233 seconds across the four scenarios, whether a view held 6,397 or 5.06 million records; the two empty Scenario 3 views, with no observations to join, returned in 0.07 seconds. The similar times are consistent with the fixed ClinVar join dominating the workload, not with generally scale-independent retrieval. Scenario 2 release writing took 59–81 seconds per requester, compared with 20–27 minutes in pre-release builds. Three changes reduced runtime and left the views byte-identical: the selections of all applicable rules are merged once, input files are filtered in parallel, and each resource's ancestor IRIs, which determine whether a selection covers it, are found once rather than once per rule. Scenario 3 peaked at 11.2 GB of host memory in use and 31 GB of disk. For Scenario 4, conversion required 88.7 minutes, linking 16.5 minutes, and each selective view 55–70 minutes to write.

Table S11. Stage costs by scenario and requester, underlying Figure S4. Conversion and linking run once per scenario and include ClinVar. Conventional gives the conventional workflow's single run for all requesters, including container startup; neither workflow's times include the preprocessing they share. Write covers policy evaluation and streaming the release view. Check covers release validation, including a separate index for non-empty views. Check is measured as the interval before the next requester's evaluation and is therefore unavailable for the last requester. Index covers query-index construction, and Query reports the median of three runs on the index. The Records column counts release-view records. The Index triples column includes ClinVar and links. Abbreviations follow Table S8.

Requester	Records	Index triples	Write *min*	Check *min*	Index *min*	Query *s*
Scenario 1: five gene-span slices: convert 4.2 min, link 1.5 min; conventional 4 s						
All (no policy)	51,386	42,543,789	—	—	2.3	177
CC	19,625	37,514,230	0.15	0.50	2.2	176
DS	23,933	37,813,325	0.13	0.53	2.2	171
GRU	16,105	35,967,049	0.13	—	2.2	173
Scenario 2: cohort of 104: convert 34.1 min, link 22.2 min; conventional 77 s						
All (no policy)	1,150,288	110,268,925	—	—	3.2	203
CC	421,991	65,787,093	0.98	3.0	2.5	180
DS	685,928	81,878,625	1.4	4.9	3.1	188
GRU	356,835	61,794,418	1.2	—	2.4	179
Scenario 3: complete HG005 VCF: convert 55.3 min, link 8.1 min; conventional 54 s						
All (no policy)	3,887,810	701,489,559	—	—	45.9	217
CC	3,887,810	701,489,559	38.9	70.0	46.5	217
DS	0	33,428,173	28.5	1.1	2.3	0.07
GRU	0	33,428,173	28.7	—	2.3	0.07
Scenario 4: complete NB72462M VCF: convert 88.7 min, link 16.5 min; conventional 91 s						
All (no policy)	5,063,417	1,255,334,983	—	—	85.7	232
OP	5,063,417	1,255,334,983	69.7	124.3	86.6	233
CC	5,063,411	1,255,333,543	69.7	124.8	86.5	232
DS	6,397	34,971,594	54.5	11.0	2.3	187
GRU	5,060,897	1,254,725,280	69.4	—	86.2	226

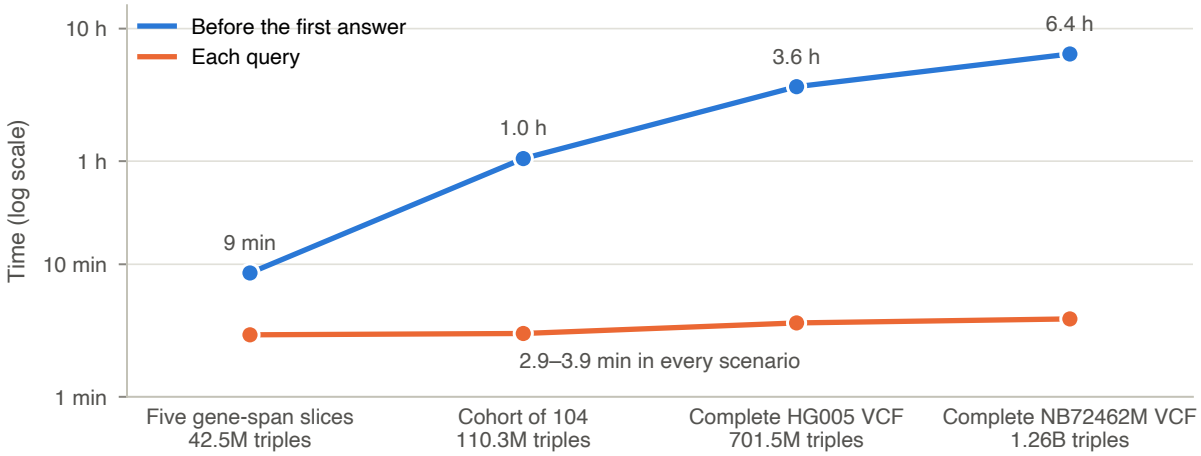


Figure S4. Most linked-workflow cost occurs before the first query. The upper line combines scenario-level conversion and linking with clinical-care view writing, validation, and indexing before the first answer. The lower line is the median query time. The indexed size includes each scenario's VCF graphs, ClinVar, frequencies where applicable, and linksets.

S6 Reproducibility and execution

S6.1 The core benchmark

The *core benchmark* produced the main text's fidelity and scalability results. It contains 143 runs (*cells*) in 12 experiment families, executed with the published VCF-RDFizer v3.1.0 release on two identical hosts, `vcf-bench-1` and `vcf-bench-2` (Sections S6.2 and S6.3). Each family corresponds to one numbered script in the testing repository's benchmark harness. The harness and the run records of every reported result are archived on Zenodo as version v1.0.0 (`https://doi.org/10.5281/zenodo.23301186`; all versions: `https://doi.org/10.5281/zenodo.23301185`). Scripts `00` and `02` record the environment and derive scaling inputs and are excluded from the count. Each complete family ran on one host without concurrent benchmarks. Every cell recorded the command, commit, image, exit status, and timings in a fresh output directory. The experiment families are defined below.

Records scaling (`04`). HG005 prefixes contain 10,000, 100,000, and one million records, each with three replicates. The complete VCF (3,856,856 records) was processed once, giving 10 cells in total. Results appear in main Section 3.3 and Figure S5.

Sample representation (`03`). Both sample profiles (Section S3.1) process 10,000 records from the 1000 Genomes chromosome-20 call set at 1, 4, 16, 64, 256, 1,024, and 2,504 samples (14 cells). Three timing replicates at 1 and 2,504 samples under both profiles add 12 cells. Four anchor cells cover two complete files under both profiles, the 10,000-line single-sample fixture and a 2,504-sample fixture, to check the same contrast outside the derived slices; the expanded 2,504-sample cell was skipped (below). Results appear in main Section 3.3.

Storage mode (`01`). Plain storage writes uncompressed N-Triples, whereas space-optimized storage streams each output part into gzip-framed N-Triples and deletes the part, so no full uncompressed copy exists. The two modes are compared on 100,000 HG005 records with three replicates per mode. Each mode also processes a 1,155,741-record prefix of the `NG131FQA1I` consumer WGS VCF (269.3 million triples) once, giving 8 cells in total. Results appear in main Section 3.3 and Figure S5f.

Corpus breadth (`05`). HDT and COTTAS are built from the first 250,000 records of eight public inputs and from the complete HG005 VCF; a tenth cell, the complete 1000 Genomes call set, was skipped by the cohort-scale guard (10 cells). Results appear in main Section 3.3 and Figure S7.

Representation axes (`07`). Raw and structured INFO are compared on 100,000 HG005 records and HGSVC2 with three replicates per setting. Other cells compare basic with structured header output and convert fixtures that declare VCF 4.1 to 4.5 or no version. The family contains 20 cells (Sections S7 and S10).

Difficult inputs (`09`). The family includes 11 difficult-input fixtures (11 cells), a 100-record synthetic fixture linked with the three example linkers distributed since v3.0.0 (`rsid-dbsnp`, `gene-demo` with synthetic gene intervals, and the live `rsid-ensembl`; 86, 0, and 0 links; 4 cells), and a custom-mapping cell (1 cell), 16 cells in total. The SPDI, Ensembl-gene, and MyVariant linkers used in the linked workflow were added in v3.3.0. The sites-only cell was rerun with a corrected fixture (Section S8). Results appear in Section S7 and Table S15.

Covering set (`11`). The 10 configuration rows on a 1,000-line fixture use every value of seven options at least once, and every allowed pair of values from any two options appears together in at least one row; the full set of combinations is not tested. Each row is validated with shapes, giving 10 cells (Section S10).

Modes (`12`). Operating modes are tested independently on the 1,000-line fixture and derived artifacts (7 cells). The modes are TSV, conversion, validation, compression, decompression,

HDT indexing, and COTTAS indexing (Section S10).

Feasibility (10). One million HG005 records are processed under three configurations with requested memory ceilings of 8, 16, and 31 GB. The family contains 9 cells (Section S10).

Robustness (08). Tests cover two identical conversions of 100,000 HG005 records, a convert–compress–decompress round trip, and an index build and rebuild. The family also measures detection of 113 injected faults with the default profile and with all three profiles. The family contains 9 cells (main Section 3.1 and Section S4).

Equivalence (06). The 10,000-line (9,986-record) fixture is converted under both storage modes and both sample profiles. Validation covers four SPARQL engines, three RDF artifacts, and two HDT construction strategies: `partitioned`, which builds HDT from chunks and merges the parts, and `single`, which builds it in one `rdf2hdt` pass over uncompressed N-Triples. Two further cells test configurations the tool must refuse. The family contains 8 cells (Section S10).

Query cost (13). The 13 source-comparison queries are compared with cyvcf2 on the 10,000-line fixture (0.96M triples) and 100,000 HG005 records (17.1M triples). Three replicates per input give 6 cells (main Section 3.3 and Section S9.4).

Five families contribute the 78 validation runs: Difficult inputs (20), Covering set (23), Equivalence (16), Query cost (18), and Modes (1). A cell contains one validation run per artifact it validates (N-Triples, HDT, or COTTAS), so a family can have more validation runs than cells.

The core benchmark took 96.17 h of measured wall time (Table S12). Every asserted expectation held, but not every cell completed successfully. The truncated-gzip fixture exited non-zero, and the exit was recorded as an outcome. Two cells required refusal of the single-HDT strategy with COTTAS or space-optimized storage. Three cells were skipped. Two skips involved a 2,504-sample input under the expanded profile: the complete 1000 Genomes call set in Corpus breadth and the multi-sample anchor in Sample representation. Both are complete 2,504-sample files, which the harness's cohort-scale guard skips under the expanded profile; the sample-count series uses 10,000-record slices and is not guarded. The custom-mapping cell was skipped because no custom rules file was supplied. The two fault-injection cells record the host-side test-harness commit rather than a conversion-container execution.

Experiments outside the core benchmark are reported separately and excluded from its totals (Table S13). Separate experiments cover the linked workflow, regional retrieval with tabix, and large-graph retrieval on `vcf-bench-3`. Two additional validation experiments comprise a default-profile fault-injection rerun and a consumer WGS run on 250,000 `NG131FQA1I` records. The consumer WGS run was repeated with the corrected v3.3.1 validator. Other separate experiments are the N-Triples-only rerun of the 100,000-record query-cost workload, the index-storage measurement, and the shared-input converter comparison. The earlier pre-release benchmark runs and host-calibration runs are also excluded. The testing repository's `legacy` branch archives the earlier runs separately from the reported results.

S6.2 Builds and dependencies

The core benchmark used VCF-RDFizer v3.1.0, commit `d3b34d5`. The Docker image is `ecrum19/vcf-rdfizer:v3.1.0`, with digest

`sha256:1904e96dde12ab2e2e70d8ee1267765c293ab100a8bd1b14d5b009b2bf8e34aa`

Image rebuilding was disabled during execution. The image bundles VCF Core SHACL shapes from vocabulary commit `5bfd19d`. The bundled snapshot differs from the separately versioned vocabulary release [19].

Table S12. Core benchmark by experiment family. Outcome columns count cells marked OK, recorded (Rec.), refused (Ref.), or skipped (Skip.). Recorded indicates a non-zero exit without an asserted outcome, refused indicates an expected rejection, and skipped indicates that a guard or a missing input prevented execution. The truncated-gzip cell counts as recorded because the harness accepts either conversion or a diagnosed refusal of a difficult input and asserted neither in advance; Section S7 reports the refusal as intended. Hosts b1 and b2 are `vcf-bench-1` and `vcf-bench-2`. Section references without an S prefix refer to the main text.

				Outcome				
Experiment	**Host**	**Cells**	**Hours**	**OK**	**Rec.**	**Ref.**	**Skip.**	**Question answered**
Conversion, scaling, and storage								
Records scaling	b1	10	10.99	10				Cost growth with records (§3.3)
Sample representation	b2	30	19.69	29			1	Profile growth with samples (§3.3)
Storage mode	b1	8	12.87	8				Peak workspace, plain vs space-optimized (§3.3)
Corpus breadth	b2	10	26.90	9			1	HDT and COTTAS on 10 public inputs (§3.3)
Input and configuration coverage								
Representation axes	b1	20	4.82	20				VCF 4.1–4.5, INFO, and header costs (§S7, §S10)
Difficult inputs	b1	16	0.11	14	1		1	11 difficult-input fixtures and demonstration linkers (§S7)
Covering set	b1	10	0.35	10				Every option value and allowed pair (§S10)
Modes	b1	7	0.03	7				Each operating mode alone (§S10)
Feasibility	b1	9	9.03	9				Memory ceiling × configuration (§S10)
Validation and retrieval								
Robustness	b1	9	0.25	9				Determinism, round trip, fault injection (§3.1, §S4)
Equivalence	b2	8	6.84	6		2		Four engines, both profiles, three artifacts (§S10)
Query cost	b1	6	4.27	6				SPARQL vs cyvcf2 on identical work (§3.3)
Total		**143**	**96.17**	**137**	**1**	**2**	**3**	

The image pins RMLStreamer 2.5.0, hdtc 1.1.0, pycottas 1.1.0, DuckDB 1.5.5, cyvcf2 0.34.0, and pyshacl 0.30.1. Query engines are Comunica 5.3.0 for N-Triples, `@comunica/query-sparql-hdt` 5.0.1, the COTTAS Comunica engine 0.1.0, and QLever pinned by image digest. The image uses bcftools from the base distribution. The image also fixes the hdt-cpp revision used to compile `rdf2hdt` and `hdt2rdf`. Later experiment families are identified separately in Table S13. All results first produced by pre-release builds outside the core benchmark were rerun with published VCF-RDFizer v3.3.1, commit `b25fb7b`. The image digest is

`sha256:3ad71b1a54612142e3be43b24e7e4a38551949cb641f4421bcae9bf051102993`

Harness commit `6f2239ff` reran the four use-case scenarios, regional and large-graph retrieval, and consumer WGS validation using the same inputs. Each experiment used the original host except Scenario 4, which moved from `vcf-bench-3` to `vcf-bench-2`. Retrieval reused the v3.1.0 graphs. Converting the same 100,000 HG005 records with v3.3.1 produced 17,098,746 sorted triples identical to the v3.1.0 output. The MyVariant.info tier did not contact the service again. Using harness commit `4ea07cc7`, v3.3.1 replayed the 21 responses recorded during the pre-release run on 2026-09-28 and confirmed the same links.

The 657.4M-triple conversion/retrieval graph and the 662.4M-triple normalized use-case graph are different artifacts, not alternative counts of the same graph.

Table S13. Builds and scope of each experiment family. Recorded state identifies the software build and, where stated, the host; Scope lists the inputs and checks the family covers. A reproducible state requires the recorded code, local changes, dependencies, and external resources.

Family	Recorded state	Scope
143-cell core benchmark	v3.1.0, `d3b34d5`, with the full image digest above	Conversion, artifacts, fixtures, 1,006 applicable paired comparisons, and configuration tests
Regional retrieval	v3.3.1 (image `sha256:3ad71b1a...2993`) on the v3.1.0 query-cost graphs, using `vcf-bench-1`	Runner with tabix, 11,160 regional executions
Large-graph retrieval	v3.3.1 on the v3.1.0 graphs, using `vcf-bench-3`	Separate host, 13 summary queries without SHACL
Index storage	v3.3.1 (image `sha256:3ad71b1a...2993`) on the query-cost and large-graph N-Triples, using `vcf-bench-3`	bgzip and tabix, and QLever index sizes, for 100,000 HG005 records and the complete VCF
Release conversion check	v3.3.1 against v3.1.0, on `vcf-bench-2`	100,000 HG005 records produced identical sorted triples. The v3.1.0 graphs therefore represent v3.3.1 output
Linked use case	v3.3.1 for every stage. Scenarios 1–2 ran on `vcf-bench-1` and Scenarios 3–4 on `vcf-bench-2`	SPDI, gene links, policy views, and record-level comparison. The MyVariant.info tier replays responses recorded on 2026-09-28
Additional validation runs	v3.1.0 (`d3b34d5`, image `sha256:1904e96d...34aa`) on `vcf-bench-2`	Default-profile fault-injection rerun and consumer WGS validation (250,000 NG131FQA1I records). The consumer run used QLever and expanded output; the graph passed v3.1.0's file-size-based SHACL guard but was too large for whole-graph shape validation (Table S16), so it was repeated without shapes, and the v3.3.1 rerun with shapes is the next row
Consumer WGS validation rerun	v3.3.1 on `vcf-bench-2`	Same slice and graph, QLever, and expanded profile. Default shapes ran in 117 batches on four workers
N-Triples-only rerun	v3.1.0 (`d3b34d5`, image `sha256:1904e96d...34aa`) on `vcf-bench-2`	100,000 HG005 records converted to gzip-framed N-Triples only. Query-cost validation used QLever without SHACL, with three replicates of each stage
Shared-input converter comparison	Harness `e3743590` and VCF-RDFizer v3.3.1 image `sha256:3ad71b1a...2993`. Other converter builds follow Table S2. All ran on `vcf-bench-1`	100,000 HG005 records and the 16-sample, 10,000-record subset of the sample-count series. Q1–Q8 were compared across three replicates (Section S2)

S6.3 Hosts, measurements, and execution status

The core benchmark used two separate hosts, `vcf-bench-1` and `vcf-bench-2`, with identical specifications. Each host had an Intel Xeon Silver 4309Y at 2.80 GHz (8 cores), 33.6 GB RAM without swap, and a 210 GB SSD. Both hosts ran Ubuntu 24.04.4 LTS, kernel 6.8.0-85-generic, Docker 29.7.2, and Python 3.12.3. Each complete experiment ran on one host without concurrent benchmark jobs. As a calibration before the core benchmark, each host ran the Modes family (`12`) on the 1,000-line fixture with identical settings. The two runs produced identical triple sets and byte-identical COTTAS artifacts and HDT indexes. N-Triples and HDT differed only in ordering and the HDT `dcterms:issued` timestamp. Conversion took 26 s on `vcf-bench-1` and 37 s on `vcf-bench-2`, and validation took 156 and 208 s, respectively. Timings are therefore compared within hosts. Large-graph retrieval ran separately on `vcf-bench-3`, with the same CPU, 8 cores, 33.6 GB RAM without swap, Ubuntu 22.04.5, and kernel 5.15. The large-graph experiment queried v3.1.0 graphs with the v3.3.1 runner.

Each cell records the command, host, commit, image, exit status, and wall time in a fresh output directory. Stage reports record resident memory, triple counts, artifact sizes, validation outcomes, and transient workspace separately. Performance denominators use uncompressed VCF bytes. Table S14 reports compressed download sizes. Replicated comparisons describe variability, whereas single-run corpus results do not estimate run-to-run variation.

The testing repository contains commands, normalized answers, and comparison reports. A persistent archive of the complete evaluated snapshots remains outstanding. Tagged releases are distributed through PyPI, conda-forge, and Docker Hub under the MIT License, with installation instructions, examples, release notes, and issue tracking [47]. The unit-test suite covers Linux, macOS, and Windows. All reported benchmarks used Ubuntu.

S7 Input and version coverage

Table S14 identifies the 10 public inputs. The testing repository provides download commands and canonical filenames.

Table S14. Public corpus and evaluation roles. Names are canonical `.vcf.gz` stems. Assemblies and sample counts were checked against the headers. Sizes are approximate compressed download sizes. WGS denotes whole-genome sequencing. Each file contains variant calls, not a genome sequence.

File	Provider	Assembly	Samples	MB (gz)	Evaluation role
`NG1N86S6FC`	Sequencing.com	GRCh38	1	379	Consumer WGS
`NG131FQA1I`	Dante Labs	GRCh38	1	224	Consumer WGS
`NB72462M`	Nebula Genomics	GRCh38	1	341	Consumer WGS
`60820188475559`	PGP upload	GRCh37	1	325	SNP-only profile
`60820188474283`	Dante Labs	GRCh37	1	222	Consumer WGS
`OGOOR_HG002`	precisionFDA	GRCh38	1	69	GIAB Truth Challenge v2
`1000G_phase3_chr20`	1000 Genomes	GRCh37	2,504	327	Phased cohort
`HGSVC2`	HGSVC2	GRCh38	32	31.5	Multi-sample SVs
`HG004_GRCh38`	GIAB v4.2.1	GRCh38	1	149	Benchmark VCF, chr1–22
`HG005_GRCh38`	GIAB v4.2.1	GRCh38	1	139	Benchmark VCF, chr1–22

Versions and corpus processing. Fixtures declaring VCFv4.1–4.5, plus one with no `##fileformat`, all converted. The generated RML mappings differ only in the file class, from `vcfc:VCF41File` to `vcfc:VCF45File`. Version-specific fields therefore come from the direct emitters, which read the header declarations and record values (Section S3.2). The two-record VCFv4.5 fixture contains `LAA` and `LAD`, the local-allele FORMAT fields of VCF 4.5, while the VCFv4.4 fixture contains the phase-set field `PS`. The fixtures produce 252.5 and 210 triples

per record, respectively. The results demonstrate field retention, not interpretation of every version-specific construct. Local-to-global allele resolution through `LAA` is implemented for the expanded profile, but the fixture does not exercise it, because it declares `LAD` with `Number=R` rather than `Number=LR`. The undeclared-version input uses the documented fallback.

Nine corpus inputs were processed with headers retained: the first 250,000 records of eight files, and HG005 whole. All corresponding HDT and COTTAS outputs passed decoding/count checks. The harness's cohort-scale guard skips expanded conversion of files with more than 1,000 sample columns. The harness therefore skipped the complete 2,504-sample 1000 Genomes input. Both sample profiles were instead tested on 10,000-record slices containing all 2,504 samples. The tool separately refuses expanded conversion when estimated workspace exceeds three quarters of free space and suggests condensed output. Neither guard is evidence that the complete cohort was converted.

Difficult inputs. Ten of the 11 fixtures converted and completed paired validation (Section S4.1). The truncated gzip was refused during decompression, as intended (Table S15). The sites-only result comes from a rerun after correcting the fixture, which was malformed in the core benchmark. The original failure arose from the fixture rather than the tool (Table S16). Genotype queries Q5 and Q6 were verified as not applicable because the sites-only file has no samples.

Table S15. Difficult-input fixtures. Triple counts describe the produced graph. An intended refusal is not a fidelity result.

Fixture	Triples	Validation	Note
CRLF line endings	262	PASS	No stray `\r` in the last field's literal
Symbolic alleles	453	PASS	`<DEL>` not treated as a base string
Breakend ALT	417	PASS	Breakend ALT syntax preserved
Mixed ploidy	350	PASS	Tetraploid and triploid calls kept their source ploidy
Half and haploid calls	446	PASS	Four genotype cases stayed distinct
Missing values	321	PASS	Missingness follows the field-specific representation
FILTER values	486	PASS	`PASS`, `.`, and a failed code are three states
Declared FORMAT unused	293	PASS	Header declaration represented although never used
Zero data records	189	PASS	Header-only input
Sites only	183	PASS	No FORMAT or sample columns. Rerun with a corrected fixture
Truncated gzip	—	refused (intended)	Refused at decompression in 0.58 s with a diagnostic

S8 Defects, corrections, and remaining gaps

Table S16 records defects and incomplete checks that are not visible in aggregate pass counts. Corrected outputs are used where stated. The testing repository's `legacy` branch archives unsuccessful, superseded, and pre-release runs separately from the reported results.

Table S16. Defects, diagnostic evidence, and current status.

Issue	Evidence	Correction or remaining limit
Header-only samples	Missing `SampleSet` and `VCFSample` resources in a file with declared samples but no records. *Found by Q9 and Q10.*	Sample resources are now derived from the header, increasing the fixture from 181 to 189 triples. The preflight oracle now expects no per-record calls.

Table S16, continued

Issue	Evidence	Correction or remaining limit
Dangling allele references	Raw INFO with expanded samples produced 1,972 genotype references to absent allele resources and 5,916 shape violations in the 1,000-line (986-record) covering-set fixture. *Found by SHACL shapes; Q9 and Q10 passed because their expectations shared the emitter's rule.*	Allele resources are emitted whenever required by INFO or sample representation. Census expectations and regression tests cover all four INFO/sample combinations.
Withheld-file metadata	The policy plug-in removed the correct records but kept the header of a withheld file. *Found by the policy plug-in's release check.*	The plug-in now removes a withheld file together with the resources that depend on it, such as its header lines.
Shape memory guards	A 170,935,101-triple graph passed the size guard as 390.7 MB COTTAS and exhausted 33.0 GB. Larger packaged forms of the same graph were skipped. A 58.2M-triple graph also passed the guard based on the 63.8 MB VCF size.	Version 3.3.0 uses triple counts for the guard. Version 3.3.1 validates record-local default shapes in batches. Deeper cross-record profiles retain a whole-graph limit.
Comunica heap limits	Comunica-backed endpoints exhausted the JavaScript heap while approximately 25 GB of host memory remained.	The heap ceiling is configurable. Failed runs remain archived.
COTTAS query execution	The original pycottas/RDFLib path stalled on Q5 for approximately 41 and 10 hours. In-process timeouts failed to interrupt the query.	The DuckDB-backed Comunica path reduced Q5 to 2–5 seconds and the 13-query workload to 38 seconds. Completed comparisons use the replacement path.
SHACL severity and missing tokens	The report parser treated non-blocking SHACL results (`sh:Warning`, `sh:Info`) as failures, and the rule for VCF's missing-value token (`.`) flagged legitimate dots, such as `Number=.` declarations and fully missing genotypes written as `..`	Only `sh:Violation` blocks a run, and legitimate dot uses are excluded. Nine flags on valid output remain in the core benchmark's v3.1.0 run records.

Table S16, continued

Issue	Evidence	Correction or remaining limit
Sites-only fixture	The generator declared a FORMAT column without samples, which bcftools rejects. Conversion produced 245 triples and HDT, but neither paired validation completed.	The generator was corrected, and the cell was rerun with the same release. The rerun produced 183 triples and two passing paired validations. VCF-RDFizer had accepted the malformed header without a warning.
Legacy measurements	An earlier benchmark under-reported partitioned compression time, labeled uncompressed input bytes as `.vcf.gz`, and compared HDT with uncompressed rather than gzip-framed N-Triples.	Timing was corrected in commit `fb48e50`. The current analysis excludes the legacy tables.
Consumer WGS oracle	Version 3.1.0 omitted phase sets and compared lexical QUAL forms, producing three validator discrepancies although direct record fields agreed.	Version 3.3.1 corrected phase-set expectations and QUAL normalization. Oracle expectations also cover structural-variant events, confidence intervals, and gVCF blocks. Local-allele semantics remain outside the oracle.
Gene links for alleles without explicit bases (`*`, symbolic, breakend)	Version 3.3.0 omitted gene links for spanning deletions (`*`). Symbolic alleles and breakends also lack links. Gene-based restrictions cannot select records that carry no gene link. In Scenario 2, v3.3.1 therefore released 110 symbolic structural-variant records to DS and 58 to GRU although they lie in restricted genes.	Version 3.3.1 links spanning deletions by REF span. Symbolic and breakend extents remain an explicit limitation.

S9 Conversion, storage, and retrieval measurements

S9.1 Record and sample scaling

RDF output grew approximately linearly across the HG005 record-count series (Figure S5). A power law, triples $\propto$ records^b, fitted by least squares on log-log axes gave $b = +0.999$ (95% CI $[+0.991, +1.007]$), where $b = 1$ means linear growth, with 170.5–171.9 triples per record. The same fit gave $b = +1.033$ for artifact bytes, $+1.025$ for peak workspace, and $+1.063$ for end-to-end time. End-to-end times were 40.7, 423.7, and 5,476.0 seconds for 10,000, 100,000, and one million records. Replicate standard deviation did not exceed 13.5 seconds. Mapping-stage resident memory remained between 1.0 and 1.7 GB. The record-count series built HDT but not COTTAS; HDT construction increased from 38% of total time at 10,000 records to 82% for the complete VCF.

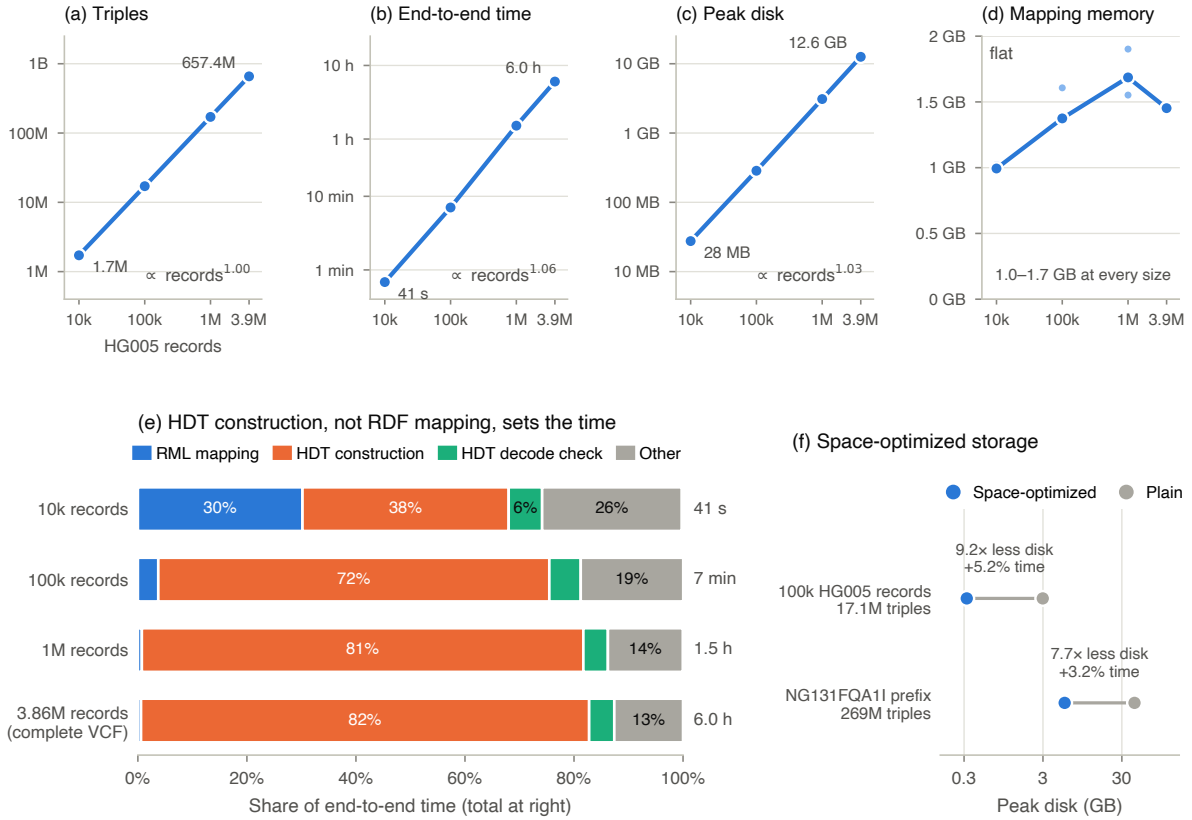


Figure S5. Record scaling and storage mode. Panels (a)–(e) report triples, time, peak disk, mapping memory, and stage shares for the HG005 record-count series. Panel (f) compares plain and space-optimized workspace. The three subsets are replicated, while the complete VCF is one run.

The sample-count series holds 10,000 records and INFO content fixed while increasing samples from one to 2,504 (Figure S6). Triple counts follow $1{,}359{,}498 + 250{,}005S$ for expanded output and $1{,}439{,}498 + 5S$ for condensed output exactly. At 2,504 samples, expanded output contains 627,372,018 triples, or 432× the condensed graph. Expanded output is 322× larger in gzip-framed N-Triples and 69× larger in HDT. Condensed HDT size grows 5.8× as sample vectors lengthen, despite only 0.9% growth in triples. Expanded HDT reaches 4.10 GB and requires 15,269 seconds, compared with 59.2 MB and 250 seconds for condensed output. Projected expanded workspace for the complete 1,812,841-record call set is approximately 2 TB, more than the 189 GB volume, so the harness's cohort-scale guard skipped the run. The projection is not a completed full-cohort measurement.

S9.2 Storage mode and optional artifacts

Plain and space-optimized storage produced identical triples. Final artifact sizes differed by no more than one filesystem block. Space-optimized mode reduced peak workspace from 2.98 to 0.325 GB on 100,000 records, with a 5.2% increase in runtime. On the same 1,155,741-record `NG131FQA1I` prefix (269.3 million triples), workspace fell from 43.40 to 5.65 GB, with a 3.2% runtime increase. Mapping time met a prespecified ±10% equivalence margin on the replicated input.

Across the corpus, COTTAS occupied 0.37–0.55 times the storage of gzip-framed N-Triples and 0.27–0.37 times the storage of HDT. COTTAS construction took 1.5–3.0 times longer than HDT construction (Figure S7). The v3.1.0 complete-HG005 run of the Corpus breadth family, which built both HDT and COTTAS, produced 657.4 million triples in 16.07 hours, including

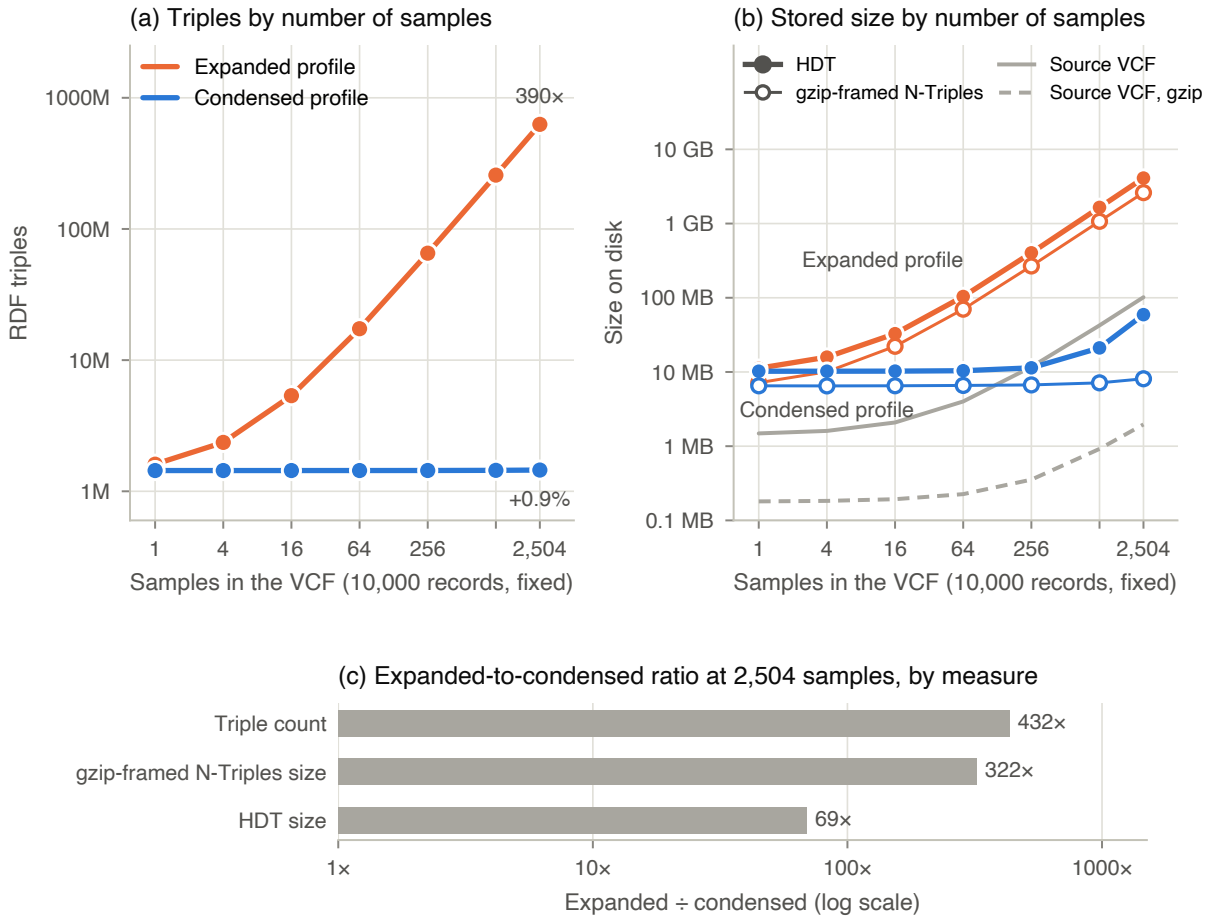


Figure S6. Sample representation trades direct addressability for size. The same 10,000 records are converted with one to 2,504 samples. Expanded resources grow with sample calls, while condensed vectors keep triple counts nearly constant but still grow in bytes.

approximately 14.5 hours for HDT and COTTAS construction. The retained HDT, HDT index, and COTTAS tree occupied 10.88 GB, while gzip-framed N-Triples alone occupied 2.89 GB. The later v3.3.1 use-case conversion produced 662.4 million genomic triples in 53 minutes, or 55 minutes with ClinVar, before linking, release processing, and indexing. The runs differ in software version, input, and processing stages. The timing difference does not measure a speed-up.

S9.3 Query workloads and physical access paths

On 17.1 million triples, QLever's 13-query totals differed by less than 1% between N-Triples, HDT, and COTTAS inputs (Figure S8). Every input path queried a QLever index rather than the original artifact. On a separate 0.96-million-triple graph, the mean total was 1.1 seconds for QLever and 23.4 seconds for Comunica over N-Triples. The COTTAS and HDT paths took 38.2 and 45.1 seconds, respectively. All engines returned identical answers.

For individual questions on 17.1 million triples, full cyvcf2 parses required 11.5–12.3 seconds, while indexed QLever required 0.005–4.62 seconds. The query-time ratios exclude conversion and the Query cost family's 22.8-second QLever index build. One cyvcf2 pass answered all 13 questions in 12.4 seconds, compared with 17.1 seconds of indexed queries, excluding setup.

The complete-HG005 workload required 646.4 seconds for all 13 questions after an 880.8-second index build. Every answer matched the parser result. The three QLever input artifacts differed by only 0.4% in indexed query time but differed in decoding time. At 171 million triples and with a 24 GB heap, the native COTTAS path answered Q1–Q5 and failed on Q6 when its engine crashed,

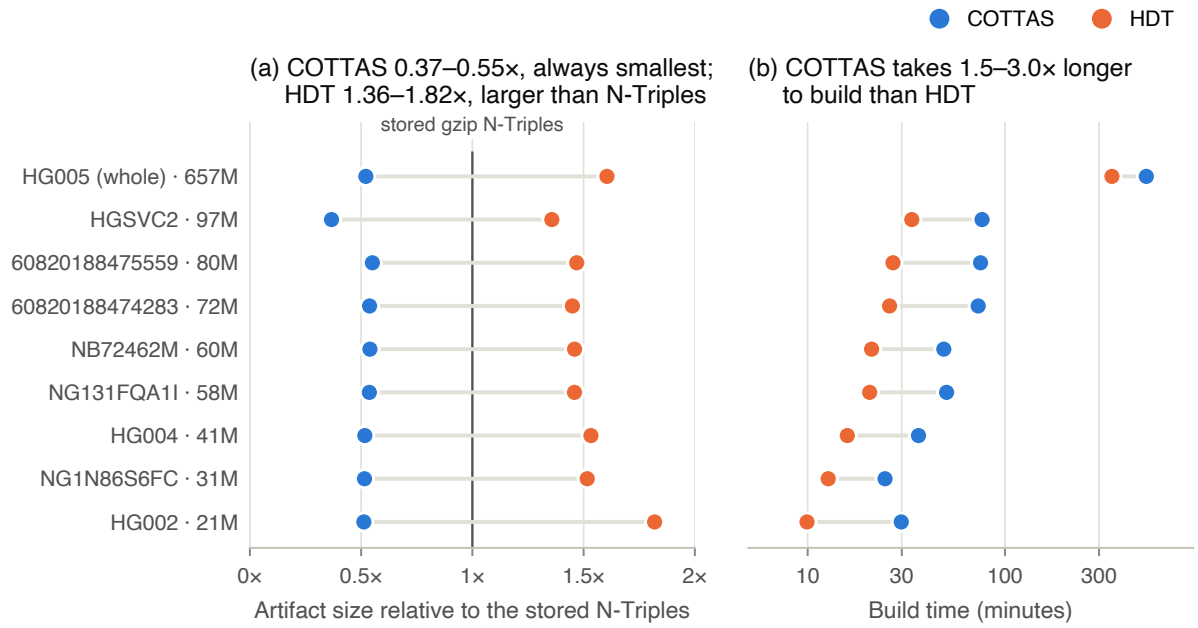


Figure S7. Corpus artifact sizes and construction times. Eight inputs use 250,000 records, and HG005 is processed completely. Artifact sizes are relative to gzip-framed N-Triples, and construction times use a logarithmic scale.

and the native HDT path answered Q1–Q4 before Q5 reached the one-hour per-query limit; in both runs the endpoint then stopped, so the remaining questions were not executed. Small-graph engine rankings therefore did not predict large-graph completion.

Table S17. Retrieval at scale on v3.1.0 HG005 graphs. The fixture and 100,000-record graphs were queried on `vcf-bench-1` with v3.1.0, and the one-million-record and complete graphs on `vcf-bench-3` with v3.3.1. Seconds per million triples do not remove host differences. QLever answers were compared with cyvcf2, and the native engines' completed answers equal QLever's. Times are sums of per-question medians; the 17.1 s quoted in the text for 100,000 records is the mean of the three replicates' 13-query totals (16.9–17.4 s). Times exclude setup (the QLever index build) and artifact decoding, which the Outcome column reports where measured.

Graph	Triples	Engine	Time (s)	Outcome
Does SPARQL scale to a complete VCF? (all 13 questions)				
Fixture	958,919	QLever	1.1	1.13 s per million triples
100 k records	17,098,746	QLever	17.0	1.00 s per million
1 M records	170,935,101	QLever	165.1	0.97 s per million
3.86 M records, complete	657,425,805	QLever	**646.4**	0.98 s per million, setup 880.8 s, and replicates within 0.6%
Does the artifact matter? (complete VCF, one engine, three packagings)				
from N-Triples		gzip-framed	646.4	Decode 184–187 s, setup 880.8 s
from HDT		—	644.8	Decode 963–979 s, setup 887.8 s
from COTTAS		—	643.9	Decode 600–608 s, setup 891.3 s
Do the other engines get there at all? (1 M records, 170,935,101 triples)				
COTTAS engine	5 of 13 answered		751.4	Q1–Q5 answered; Q6 failed after 437.3 s on a native allocation failure, and Q7–Q13 then found no endpoint
HDT engine	4 of 13 answered		8,889.5	Q1–Q4 answered; Q5 reached the one-hour per-query ceiling (3,600.1 s) without an answer, and Q6–Q13 then found no endpoint

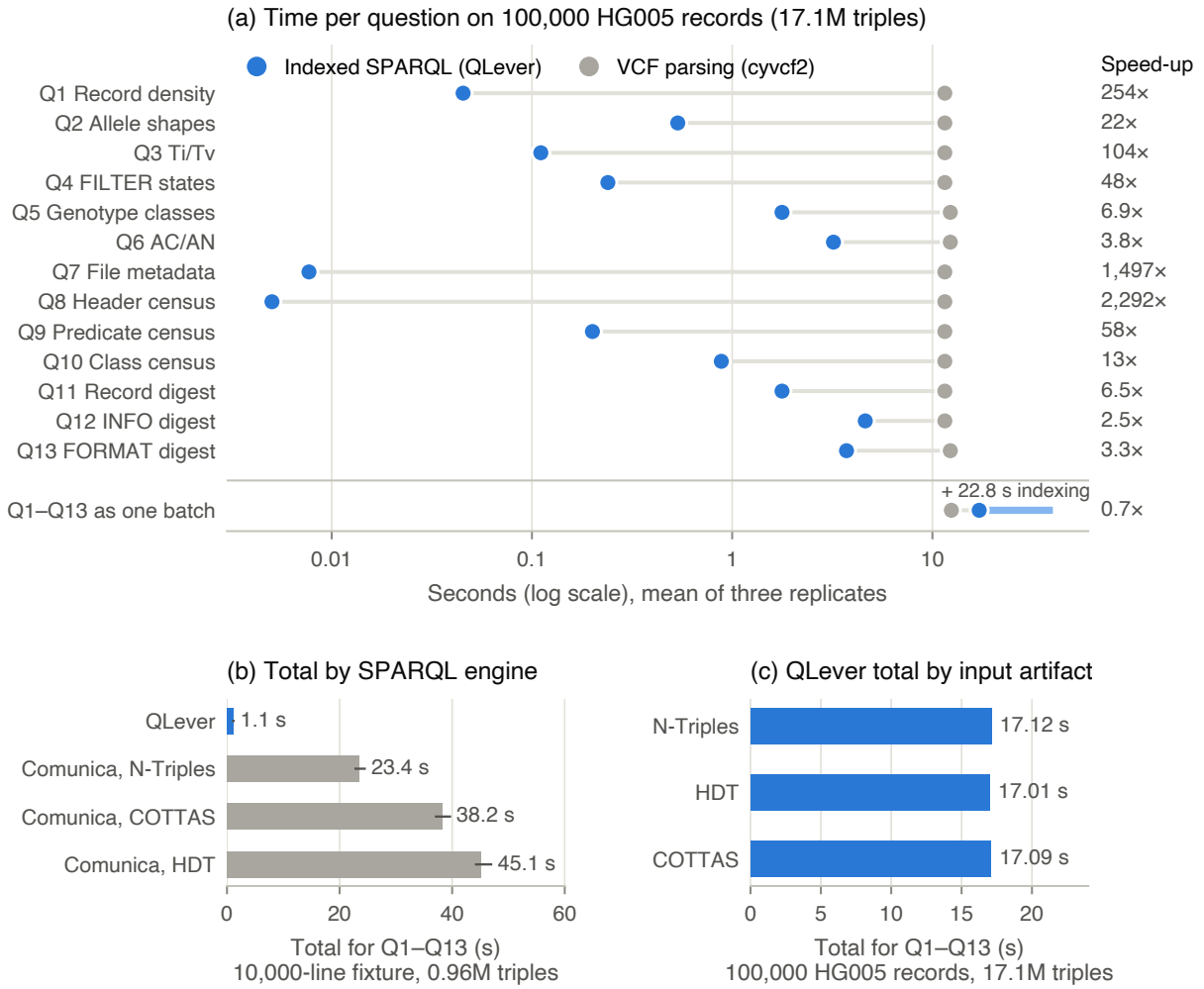


Figure S8. Detailed retrieval paths. Panel (a) compares individual questions and the batch on 17.1 million triples. Panel (b) compares four engines on 0.96 million triples. Panel (c) loads the same 17.1-million-triple graph into QLever from N-Triples, HDT, and COTTAS.

Indexed regional access. Across 11,160 regional executions, every answer agreed and no execution failed. On 17.1 million triples, QLever required 10.3–10.8 milliseconds across 1 kb–10 Mb windows. Tabix-indexed cyvcf2 and bcftools were faster for windows up to 100 kb, and bcftools also for 1 Mb windows (Table S18). Indexed-reader times increased with the number of records parsed, and QLever was faster than both readers at 10 Mb. In this run, building the QLever index from existing N-Triples took 23.3 seconds, compared with 0.46 seconds for bgzip and tabix; conversion to N-Triples is not included. The QLever index occupied 214.5 MB, 60.5 times the 3.5 MB of the bgzipped VCF plus its tabix index. For the complete HG005 VCF, the figures were 8.1 GB and 140.9 MB (57.5 times). The tabix index alone (46 KB and 1.6 MB) holds only offsets into the bgzipped VCF, which direct VCF access must also keep, so the comparison includes the VCF. Figure S9 separates the five regional questions and reports per-question break-even counts.

S9.4 Setup amortization

For an equivalent repeated workload with a positive per-question saving,

$$N_{\text{break-even}} = \frac{\text{additional RDF setup}}{\text{VCF time per question} - \text{indexed RDF time per question}}. \tag{2}$$

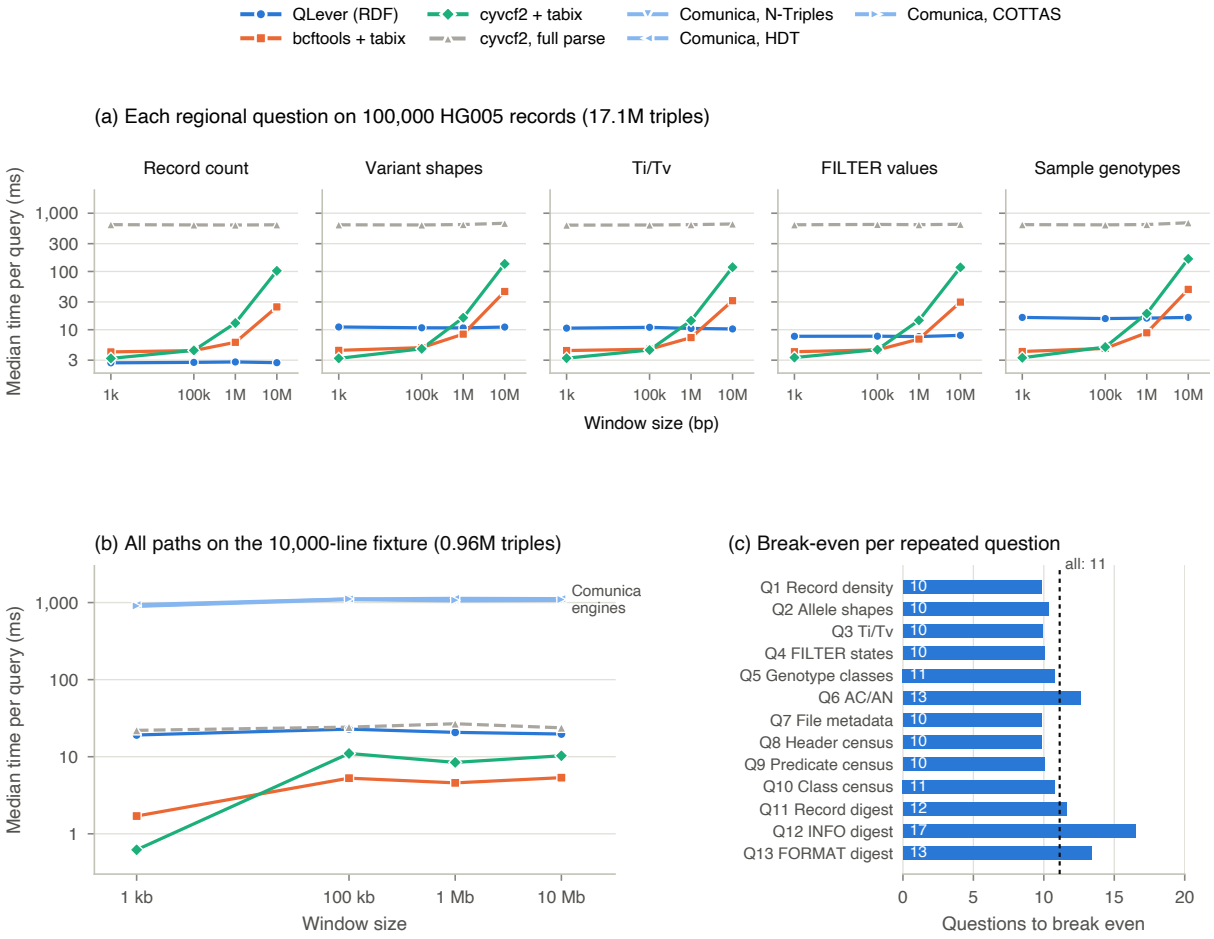


Figure S9. Regional and repeated-question retrieval in detail. Panel (a) shows each regional question on 100,000 HG005 records. Panel (b) compares seven execution paths for the regional questions on the 10,000-line fixture: QLever; Comunica over N-Triples, HDT, and COTTAS; bcftools and cyvcf2 with tabix; and a full cyvcf2 parse. Panel (c) reports the break-even repetition count for each whole-file question, including minimal RDF setup.

Table S18. Regional retrieval on 100,000 HG005 records (17.1M triples), excluding setup. The table provides the data for main Figure 7c. For each path, window size, and question, the median time is taken over all windows and three repetitions; each entry is the median of the five per-question medians, in milliseconds. QLever and the tabix-indexed readers use 20 windows per size, while full unindexed VCF parses use three. All paths use the same POS-based selection. The bottom row reports median records per window.

Execution path	1 kb	100 kb	1 Mb	10 Mb
QLever (RDF)	10.6	10.8	10.5	10.3
bcftools + tabix	4.2	4.6	7.3	31.5
cyvcf2 + tabix	3.2	4.6	14.5	118.2
cyvcf2, full parse	633.3	629.2	635.8	653.4
Records per window	*2*	*143*	*1,262*	*13,143*

A three-replicate rerun measured the minimal RDF setup, gzip-framed N-Triples and a QLever index without HDT or COTTAS, on 100,000 records. Conversion to gzip-framed N-Triples required a mean of 95.7 seconds, and QLever indexing a mean of 25.0 seconds. A full cyvcf2 parse took a median of 12.28 seconds per question, compared with a mean of 1.43 seconds per indexed query. The resulting crossover is approximately 11 separately executed questions. The core benchmark's conversion also built HDT, which QLever does not need; with its setup of 446.5 seconds (423.7 seconds of conversion and a 22.8-second index build) and its per-question times (a median of 11.57 seconds per parse and a mean of 1.32 seconds per indexed query), the estimate is 44 questions. The calculation uses the measured query mix and assumes a full VCF parse per question rather than one batched pass. Per-question crossovers range from 10 to 17 repetitions. For 100,000 records, gzip-framed N-Triples occupied 75.8 MB, compared with 3.1 MB for gzipped VCF. For complete HG005, the corresponding sizes were 2.89 GB and 139.3 MB. The experiment does not establish a universal break-even count. Costs of updating evidence or policies were not measured.

S10 Configuration coverage, operating modes, and memory feasibility

A 10-row covering set exercised every value of seven options: sample representation (expanded, condensed); INFO representation (raw, structured); storage mode (plain, space-optimized); RDF compression of the N-Triples output (gzip, Brotli); output representations (HDT, COTTAS, or both); artifact compression of the HDT and COTTAS files (gzip, Brotli); and HDT strategy (`partitioned`, `single`, or the default `auto`, which uses the partitioned path). Every row completed validation, including shapes. Pairwise coverage excludes only three prohibited combinations: `--hdt-strategy single` with space-optimized storage, COTTAS, or `hdt,cottas`. The tool refuses them because the single strategy reads one uncompressed N-Triples file, which space-optimized storage never writes, and COTTAS always requires chunked construction, so the partitioned path would run regardless. Gzip and Brotli were tested in both compression roles without comparing compression ratios or throughput.

The `tsv`, `compress`, `decompress`, HDT-index, COTTAS-index, conversion-only, and validation-only modes each ran independently with exit code 0. A feasibility sweep combined three requested memory ceilings (8, 16, and 31 GB) with three configurations (the defaults, smaller chunks, and smaller chunks with condensed samples) on one million HG005 records. All nine cells completed with 1.34–1.93 GB resident memory. The harness could not confirm that Docker limits reached every container invocation. The measurements therefore establish low memory use, not successful execution under verified memory limits.

In the Representation axes family, structured INFO increased the graph of the complete HGSVC2 structural-variant VCF from 89.01M to 96.75M triples (8.7%). N-Triples size increased by 7.4%, and HDT size increased by 7.8%. On 100,000 HG005 records, the increase was from 12.52M to 17.10M triples (36.6%). Since the dependency correction in Section S8, raw-INFO graphs still contain the allele resources that expanded sample calls reference, so the 12.52M figure includes them. Structured header output added only 0.03%: 17,098,746 versus 17,094,293 triples.

The Equivalence family used a 10,000-line, 9,986-record single-sample fixture in both sample profiles and storage modes, with N-Triples, HDT, and COTTAS artifacts. All 16 validations passed the 14 preflight checks and Q1–Q13 under all four engines. Expanded graphs contained 0.96M triples, and condensed graphs contained 0.79M triples. The complete condensed cell took 70.4 min. Two additional negative tests correctly refused single-HDT strategy with COTTAS or space-optimized storage, returning exit status 2.

References

[1] Javier D. Fernández, Miguel A. Martínez-Prieto, Claudio Gutiérrez, Axel Polleres, and Mario Arias. Binary rdf representation for publication and exchange (hdt). *Web Semantics: Science, Services and Agents on the World Wide Web*, 19:22–41, 2013.

[2] Julián Arenas-Guerrero and Sebastián Ferrada. COTTAS: Columnar triple table storage for efficient and compressed RDF management. In *The Semantic Web – ISWC 2025*, volume 16141 of *Lecture Notes in Computer Science*, pages 313–331, Cham, 2026. Springer Nature Switzerland.

[3] Nobutaka Mitsuhashi, Licht Toyo-oka, Toshiaki Katayama, Minae Kawashima, Shuichi Kawashima, Kazunori Miyazaki, and Toshihisa Takagi. TogoVar: A comprehensive Japanese genetic variation database. *Human Genome Variation*, 9(1):44, December 2022.

[4] Pierre Lindenbaum. JVarkit: java-based utilities for Bioinformatics. figshare, 5 2015.

[5] Jasmin K. Böhmer and Roel Janssen. Linked data in practice: An RDF-based approach with SPARQLing Genomics. Presentation with the SPARQLing Genomics user manual (v0.99.11), June 2020. Zenodo record, version 4.

[6] Roel Janssen. SPARQLing Genomics. Open-source software, 2019. Accessed 2026-09-03.

[7] Emanuel Diego S. Penha, Egiebade Iriabho, Alex Dussaq, Diana Magalhães de Oliveira, and Jonas S. Almeida. Isomorphic semantic mapping of variant call format (VCF2RDF). *Bioinformatics*, 33(4):547–548, 2017.

[8] Toshiaki Katayama, Shuichi Kawashima, Gos Micklem, Shin Kawano, Jin-Dong Kim, Simon Kocbek, Shinobu Okamoto, Yue Wang, Hongyan Wu, Atsuko Yamaguchi, et al. Biohackathon series in 2013 and 2014: improvements of semantic interoperability in life science data and services. *F1000Research*, 8:1677, 2019.

[9] Francesco Strozzi. bioruby-vcf2rdf: simple converter from VCF format to RDF Turtle format. GitHub repository, BioHackathon 2013, 2013. Accessed 2026-10-07.

[10] Toshiaki Katayama, Mark D. Wilkinson, Kiyoko F. Aoki-Kinoshita, Shuichi Kawashima, Yasunori Yamamoto, Atsuko Yamaguchi, Shinobu Okamoto, Shin Kawano, Jin-Dong Kim, Yue Wang, et al. BioHackathon series in 2011 and 2012: penetration of ontology and linked data in life science domains. *Journal of Biomedical Semantics*, 5:5, 2014.

[11] Joachim Baran. BioInterchange: genomic linked data converter for GFF, GVF, and VCF files. GitHub repository BioInterchange/BioInterchangeC, version 2.0.5 (commit 49d5701), MIT License, 2021. Accessed 2026-10-08.

[12] Joachim Baran, Bibi Sehnaaz Begum Durgahee, Karen Eilbeck, Erick Antezana, Robert Hoehndorf, and Michel Dumontier. GFVO: the Genomic Feature and Variation Ontology. *PeerJ*, 3:e933, 2015.

[13] Erik Garrison, Zev N. Kronenberg, Eric T. Dawson, Brent S. Pedersen, and Pjotr Prins. A spectrum of free software tools for processing the VCF variant call format: vcflib, bio-vcf, cyvcf2, hts-nim and slivar. *PLOS Computational Biology*, 18(5):e1009123, 2022.

[14] Arto Bendiken, Raoul J.P. Bonnal, and Francesco Strozzi. ruby-rdf/rdf-vcf: Rdf vcf parser, September 2015.

[15] Shivika Prasanna, Ajay Kumar, Deepthi Rao, Eduardo J. Simoes, and Praveen Rao. A scalable tool for analyzing genomic variants of humans using knowledge graphs and graph machine learning. *Frontiers in Big Data*, 7:1466391, 2025.

[16] Pierre Larmande, Bertrand Pittolat, Ndomassi Tando, Yann Pomie, Bill Gates Happi Happi, Valentin Guignon, and Manuel Ruiz. AgroLD: a knowledge graph for the plant sciences. *BMC Genomic Data*, 26(Suppl 1):73, 2025.

[17] Alexandrina Bodrug-Schepers, Hugo Chabane, Gabriela Montoya, Patricia Serrano-Alvarado, Richard Redon, and Alban Gaignard. Semantic Beacons: a framework to support federated querying over genomic variants and public Knowledge Graphs. In *Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2025)*, volume 4196 of *CEUR Workshop Proceedings*, Barcelona, Spain, February 2025. CEUR Workshop Proceedings.

[18] Hannah Bast and Björn Buchhold. QLever: A query engine for efficient SPARQL+Text search. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management (CIKM)*, pages 647–656, 2017.

[19] Elias Crum. VCF Core Vocabulary. Version 2.1.3. GitHub repository, 2026. Version 2.1.3 with the SSSOM alignment set; source snapshot of 22 September 2026.

[20] Jerven T. Bolleman, Christopher J. Mungall, Francesco Strozzi, Joachim Baran, Michel Dumontier, Raoul J. P. Bonnal, Robert Buels, Robert Hoehndorf, Takatomo Fujisawa, Toshiaki Katayama, and Peter J. A. Cock. FALDO: a semantic standard for describing the location of nucleotide and protein feature annotation. *Journal of Biomedical Semantics*, 7(1):39, December 2016.

[21] Karen Eilbeck, Suzanna E Lewis, Christopher J Mungall, Mark Yandell, Lincoln Stein, Richard Durbin, and Michael Ashburner. The Sequence Ontology: a tool for the unification of genome annotations. *Genome Biology*, 6(5):R44, April 2005.

[22] Laura Menotti, Mirco Cazzaro, Manuel Rueda, Ivo G Gut, and Gianmaria Silvello. HERO-Genomics: Bridging Genomic Data and Ontological Modelling. In *Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2025)*, Barcelona, Spain, February 2025. CEUR Workshop Proceedings.

[23] Alex H. Wagner, Reece Hart, Larry Babb, Kori Kuzma, Helen Schuilenburg, James Stevenson, Terry ONeill, Mrinal-Thomas Epic, Brad Holmes, Eric Moyer, Michael Baudis, Peter Robinson, Melissa Cline, Ronak Y. Patel, and Robert R. Freimuth. ga4gh/vrs 2.0.1, 2025.

[24] World Wide Web Consortium. Shapes constraint language (SHACL). W3C Recommendation, 2017.

[25] J. Bradley Holmes, Eric Moyer, Lon Phan, Donna Maglott, and Brandi Kattman. SPDI: data model for variants and applications at NCBI. *Bioinformatics*, 36(6):1902–1907, 2020.

[26] Renato Iannella and Serena Villata. ODRL Information Model 2.2, February 2018.

[27] Jonathan Lawson, Moran N. Cabili, Giselle Kerry, et al. The Data Use Ontology to streamline responsible access to human biomedical datasets. *Cell Genomics*, 1(2):100028, 2021.

[28] Ben De Meester, Pieter Heyvaert, and Thomas Delva. RDF Mapping Language (RML), June 2024.

[29] Sitt Min Oo, Gerald Haesendonck, Ben De Meester, and Anastasia Dimou. RMLStreamer-SISO: An RDF Stream Generator from Streaming Heterogeneous Data. In Ulrike Sattler, Aidan Hogan, Maria Keet, Valentina Presutti, João Paulo A. Almeida, Hideaki Takeda, Pierre Monnin, Giuseppe Pirrò, and Claudia d'Amato, editors, *The Semantic Web – ISWC 2022*, volume 13489, pages 697–713. Springer International Publishing, Cham, 2022. Series Title: Lecture Notes in Computer Science.

[30] RDF/HDT contributors. hdt-cpp: HDT C++ library and tools, 2026. Built from the repository head at image build time; the exact revision is fixed by the VCF-RDFizer v3.1.0 image digest.

[31] FRINK OKN. hdtc: HDT creator, 2026. Version 1.1.0, computer software, MIT License.

[32] Dennis Diefenbach and José M. Giménez-García. HDTCat: Let's make HDT generation scale. In *The Semantic Web – ISWC 2020*, volume 12507 of *Lecture Notes in Computer Science*, pages 18–33, Cham, 2020. Springer.

[33] Julián Arenas-Guerrero. pycottas: Python cottas library for compressing and querying rdf, 2026. RDFLib store backend, triple-pattern interface, and COTTAS conversion utilities.

[34] Elias Crum. Comunica SPARQL COTTAS: a Comunica engine for query evaluation over COTTAS files, 2026. Version 0.1.0; npm package `@elias.crum/query-sparql-cottas`.

[35] Petr Danecek, Adam Auton, Goncalo Abecasis, Cornelis A. Albers, Eric Banks, Mark A. DePristo, Robert E. Handsaker, Gerton Lunter, Gabor T. Marth, Stephen T. Sherry, Gilean McVean, Richard Durbin, and 1000 Genomes Project Analysis Group. The variant call format and VCFtools. *Bioinformatics*, 27(15):2156–2158, August 2011.

[36] Petr Danecek, James K Bonfield, Jennifer Liddle, John Marshall, Valeriu Ohan, Martin O Pollard, Andrew Whitwham, Thomas Keane, Shane A McCarthy, Robert M Davies, and Heng Li. Twelve years of samtools and bcftools. *GigaScience*, 10(2):giab008, 2021.

[37] Brent S. Pedersen and Aaron R. Quinlan. cyvcf2: fast, flexible variant analysis with Python. *Bioinformatics*, 33(12):1867–1869, 2017.

[38] Ruben Taelman, Joachim Van Herwegen, Miel Vander Sande, and Ruben Verborgh. Comunica: A modular SPARQL query engine for the web. In *Proceedings of the 17th International Semantic Web Conference*, pages 239–255, Cham, 10 2018. Springer International Publishing.

[39] Comunica Association. Querying local files from the command line, 2026.

[40] Comunica Association. HDT: Querying HDT files with comunica, 2026.

[41] Jiwen Xin, Adam Mark, Cyrus Afrasiabi, Ginger Tsueng, et al. High-performance web services for querying gene and variant annotation. *Genome Biology*, 17:91, 2016.

[42] David T. Miller, Kristy Lee, Noura S. Abul-Husn, Laura M. Amendola, et al. ACMG SF v3.2 list for reporting of secondary findings in clinical exome and genome sequencing: A policy statement of the American College of Medical Genetics and Genomics (ACMG). *Genetics in Medicine*, 25(8):100866, 2023.

[43] Personal Genome Project, 2005.

[44] Justin Wagner, Nathan D. Olson, Lindsay Harris, Ziad Khan, Jesse Farek, Medhat Mahmoud, Ana Stankovic, Vladimir Kovacevic, Byunggil Yoo, Neil Miller, et al. Benchmarking challenging small variants with linked and long reads. *Cell Genomics*, 2(5):100128, 2022.

[45] Marta Byrska-Bishop, Uday S. Evani, Xuefang Zhao, et al. High-coverage whole-genome sequencing of the expanded 1000 Genomes Project cohort including 602 trios. *Cell*, 185(18):3426–3440.e19, 2022.

[46] National Center for Biotechnology Information. ClinVar VCF Release (GRCh38), September 2026. Header declares `##fileformat=VCFv4.1`; accessed 8 September 2026.

[47] Rafael C. Jiménez, Mateusz Kuzak, Monther Alhamdoosh, Michelle Barker, Bérénice Batut, Marc Borg, Salvador Capella-Gutiérrez, Neil P. Chue Hong, Martin Cook, Manuel Corpas, Michael Flannery, Leyla Garcia, Josep L. Gelpí, Simon Gladman, Carole Goble, Miguel González Ferreiro, Alejandra Gonzalez-Beltran, Philip C. Griffin, Björn Grüning, Jon Hagberg, Petr Holub, Rob Hooft, Jon Ison, Sarah Jones, Florian Kaiser, Sunghwan Kim, Matthew Krafczyk, Anna Krystalli, Hilmar Lapp, Ann Lonie, Céline Marek, Michael P. Mazanetz, Aleksandra Nenadic, Wolfgang Petersen, Esther Plomp, Mikhail Pomazkov, Karthik Ram, Stian Soiland-Reyes, Alexey Sokolov, Andreas Struck, Radoslaw Suchecki, Rutger Vos, Alan R. Williams, Patrick Brack, Marta W. Muelas, Alexander D. Diehl, Jean-Baptiste Poline, Teresa K. Atwood, Alexander Mansurov, Larisa N. Soldatova, Carl Kingsford, Paul J. Kersey, Mark D. McDowall, Magne Østeras, Cristina Guijarro-Clarke, and Steve Crouch. Four simple recommendations to encourage best practices in research software. *F1000Research*, 6:876, 2017.